

Introduction to Computer Vision



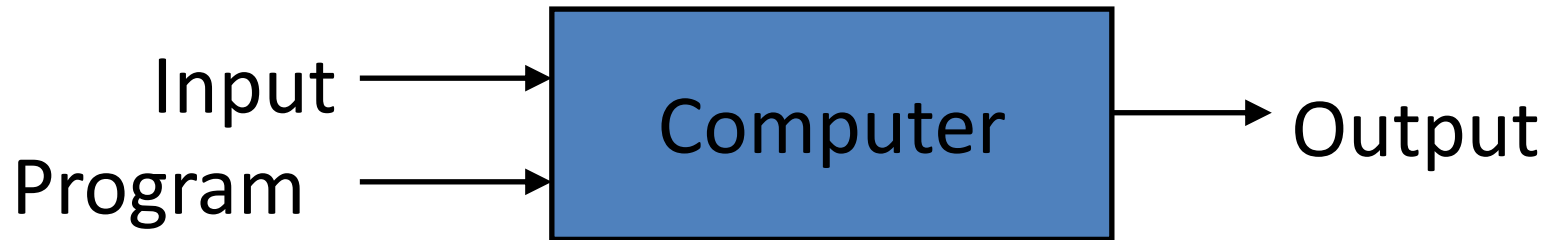
Lec.09: Deep Learning
2021.11.14

Today

- **Machine learning**
- Linear classifier
- Neural networks
- Convolutional neural networks
- Training neural networks
- Network architectures

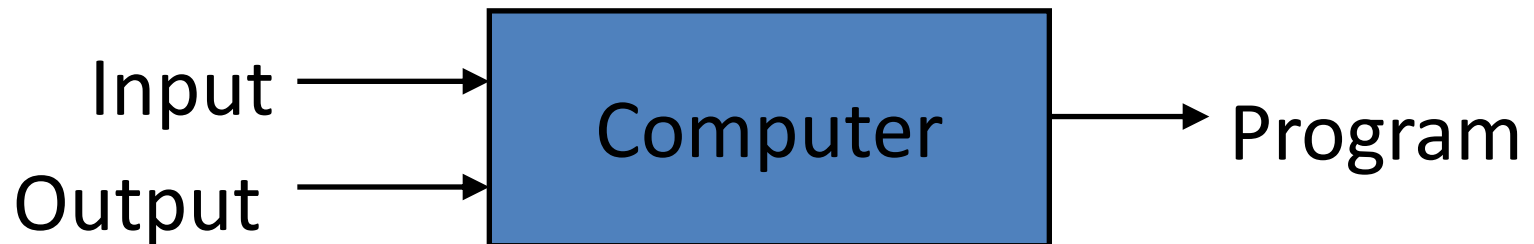
Machine learning

Traditional Programming



- Examples: image segmentation

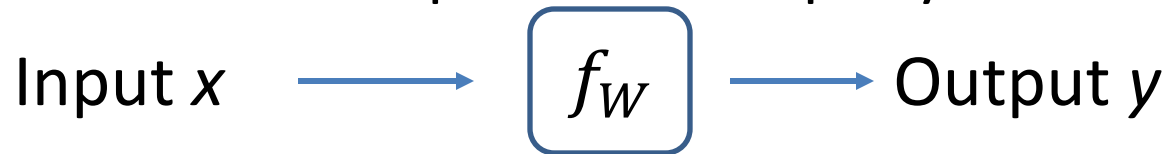
Machine Learning



- Development of computer programs that can access data and use it learn for themselves

Important concepts

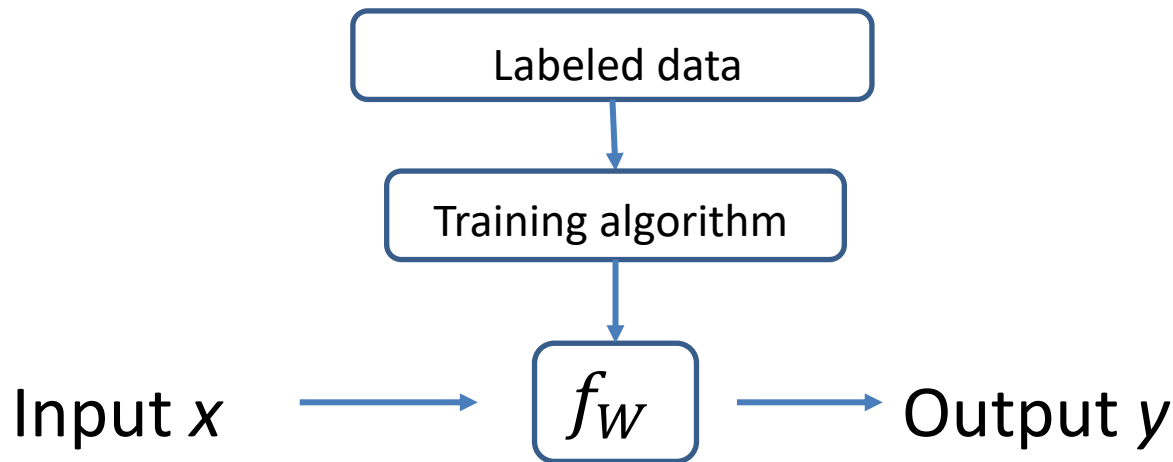
- **Model**: a mathematical description that explains the relation between input x and output y



- Defined as a function $y = f_w(x)$ with parameters denoted by w
- Examples
 - x : parents' height; y : children's height; $y = f_w(x) = w^T x$
 - x : image pixel; y : foreground/background label; $y = f_w(x) = ?$
- Problem types:
 - **Regression**: y is a real number
 - **Classification**: y is a discrete label

Important concepts

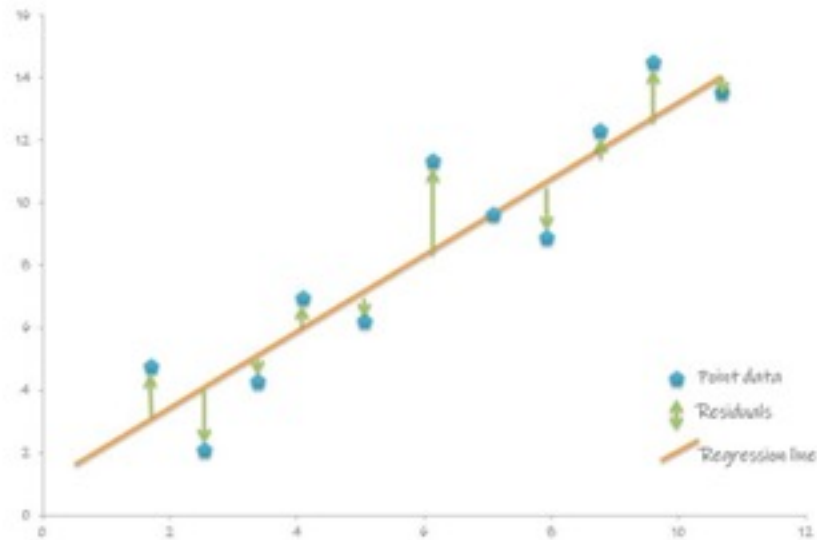
- **Supervised learning**: find f_W from labeled data
 - **labeled data**: existing (x,y) pairs, called **training data**, e.g., images already segmented



- Two phases:
 - **Training**: given labeled data, find f_W (**model fitting**)
 - **Testing**: given f_W and a new x, find y (also called **inference**)

A simple example

- Linear regression
 - What is the model?
 - Why is it called regression?
 - What is the training algorithm?



General pipeline

- Define the problem
 - What is x and y
- Collect training data: (x,y) pairs
- Design a model
 - What is the form of f_w
- Model training
 - Define a loss function $l(w)$
 - Find optimal w that minimizes $l(w)$ using some optimization algorithm
- Testing (application)
 - Given a new data x , predict y with the learned f_w

Image classification

- We consider a problem of image classification

Input: image



Output: Assign image to one of a fixed set of categories

cat
bird
deer
dog
truck



Image classification

- Easy task for human
but hard for computer



```
[[105 112 108 111 104 99 106 99 96 103 112 119 104 97 93 87]
 [ 91 98 102 106 104 79 98 103 99 105 123 136 110 105 94 85]
 [ 76 85 90 105 128 105 87 96 95 99 115 112 106 103 99 85]
 [ 99 81 81 93 120 131 127 100 95 98 102 99 96 93 101 94]
 [106 91 61 64 69 91 88 85 101 107 109 98 75 84 96 95]
 [114 108 85 55 55 69 64 54 64 87 112 129 98 74 84 91]
 [133 137 147 103 65 81 80 65 52 54 74 84 102 93 85 82]
 [128 137 144 140 109 95 86 70 62 65 63 63 60 73 86 101]
 [125 133 148 137 119 121 117 94 65 79 80 65 54 64 72 98]
 [127 125 131 147 133 127 126 131 111 96 89 75 61 64 72 84]
 [115 114 109 123 150 148 131 118 113 109 100 92 74 65 72 78]
 [ 89 93 90 97 108 147 131 118 113 114 113 109 106 95 77 80]
 [ 63 77 86 81 77 79 102 123 117 115 117 125 125 130 115 87]
 [ 62 65 82 89 78 71 80 101 124 126 119 101 107 114 131 119]
 [ 63 65 75 88 89 71 62 81 120 138 135 105 81 98 110 118]
 [ 87 65 71 87 106 95 69 45 76 130 126 107 92 94 105 112]
 [118 97 82 86 117 123 116 66 41 51 95 93 89 95 102 107]
 [164 146 112 80 82 120 124 104 76 48 45 66 88 101 102 109]
 [157 170 157 120 93 86 114 132 112 97 69 55 70 82 99 94]
 [130 128 134 161 139 100 109 118 121 134 114 87 65 53 69 86]
 [128 112 96 117 150 144 120 115 104 107 102 93 87 81 72 79]
 [123 107 96 86 83 112 153 149 122 109 104 75 80 107 112 99]
 [122 121 102 80 82 86 94 117 145 148 153 102 58 78 92 107]
 [122 164 148 103 71 56 78 83 93 103 119 139 102 61 69 84]]
```

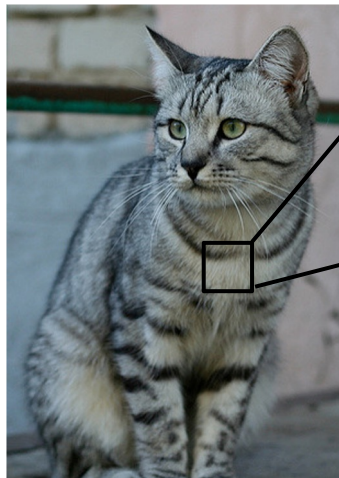
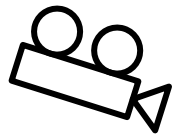
What the computer sees

An image is a big grid of
numbers between [0, 255]

e.g. 600x600x3

Image classification

- Viewpoint Variation



```
[[105 112 108 111 104 99 106 99 96 103 112 119 104 97 93 87]  
[ 91 98 102 106 104 79 98 103 99 105 123 136 110 105 94 85]  
[ 76 85 90 105 128 105 87 96 95 99 115 112 106 103 99 85]  
[ 99 81 81 93 120 131 127 100 95 98 102 99 96 93 101 94]  
[106 91 61 64 69 91 88 85 101 107 109 98 75 84 96 95]  
[114 108 85 55 55 69 64 54 64 87 112 129 98 74 84 91]  
[133 137 147 103 65 81 80 65 52 54 74 84 102 93 85 82]  
[128 137 144 140 109 95 86 70 62 65 63 63 60 73 86 101]  
[125 133 148 137 119 121 117 94 65 79 80 65 54 64 72 98]  
[127 125 131 147 133 127 126 131 111 96 89 75 61 64 72 84]  
[115 114 109 123 150 148 131 118 113 109 100 92 74 65 72 78]  
[ 89 93 90 97 108 147 131 118 113 114 113 109 106 95 77 80]  
[ 63 77 86 81 77 79 102 123 117 115 117 125 125 130 115 87]  
[ 62 65 82 89 78 71 80 101 124 126 119 101 107 114 131 119]  
[ 63 65 75 88 89 71 62 81 120 138 135 105 81 98 110 118]  
[ 87 65 71 87 106 95 69 45 76 130 126 107 92 94 105 112]  
[118 97 82 86 117 123 116 66 41 51 95 93 89 95 102 107]  
[164 146 112 80 82 120 124 104 76 48 45 66 88 101 102 109]  
[157 170 157 120 93 86 114 132 112 97 69 55 70 82 99 94]  
[130 128 134 161 139 100 109 118 121 134 114 87 65 53 69 86]  
[128 112 96 117 150 144 120 115 104 107 102 93 87 81 72 79]  
[123 107 96 86 83 112 153 149 122 109 104 75 80 107 112 99]  
[122 121 102 80 82 86 94 117 145 148 153 102 58 78 92 107]  
[122 164 148 103 71 56 78 83 93 103 119 139 102 61 69 84]]
```

All pixels change when
the camera moves!

Image classification

- Intraclass variation



Image classification

- Deformation

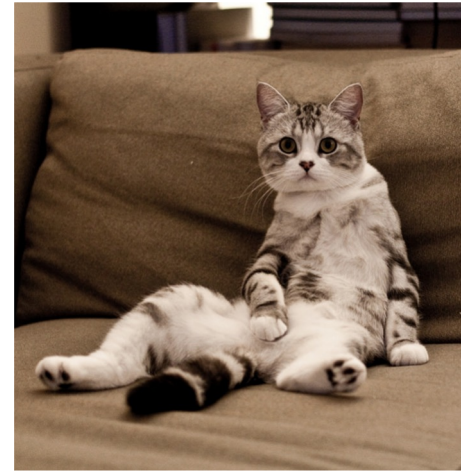


Image classification

- Illumination Changes

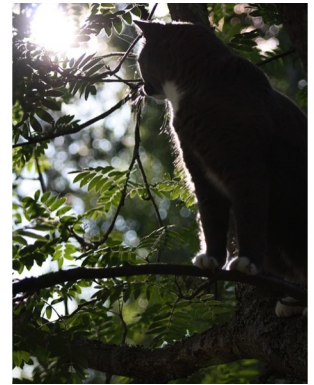


Image classification

- Background clutter



Image classification

- To classify a cat, you could try

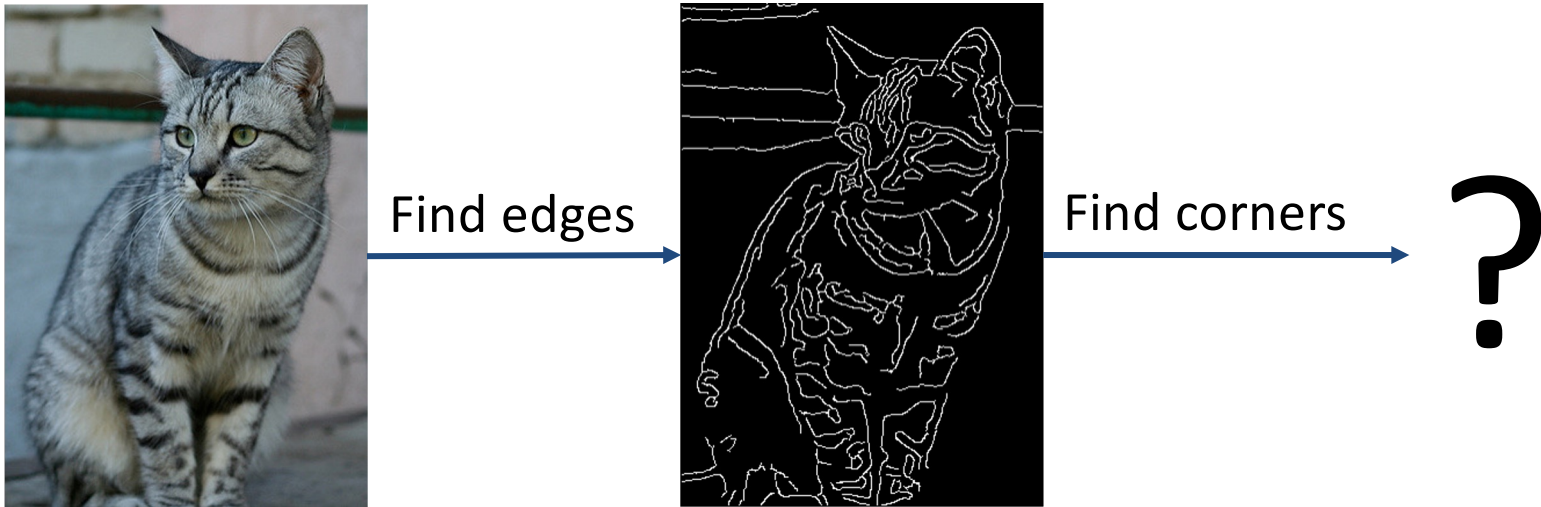


Image classification

- An Image classifier is hard to define
 - Unlike e.g. sorting a list of numbers
 - No obvious way to hard-code the algorithm for recognizing a cat or other classes

Learning to classify

- Data-driven approach
 - Collect a dataset of images and labels
 - Use machine learning to train a classifier
 - Evaluate the classifier on new images

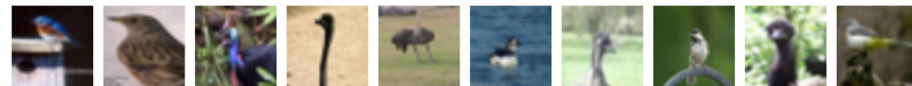
airplane



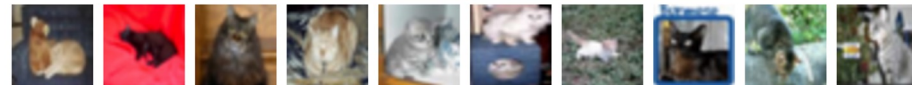
automobile



bird



cat



deer

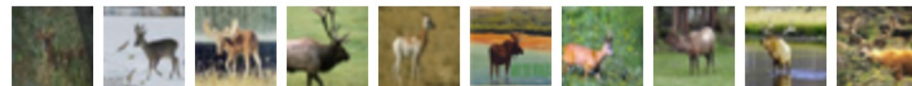


Image classification pipeline

Training

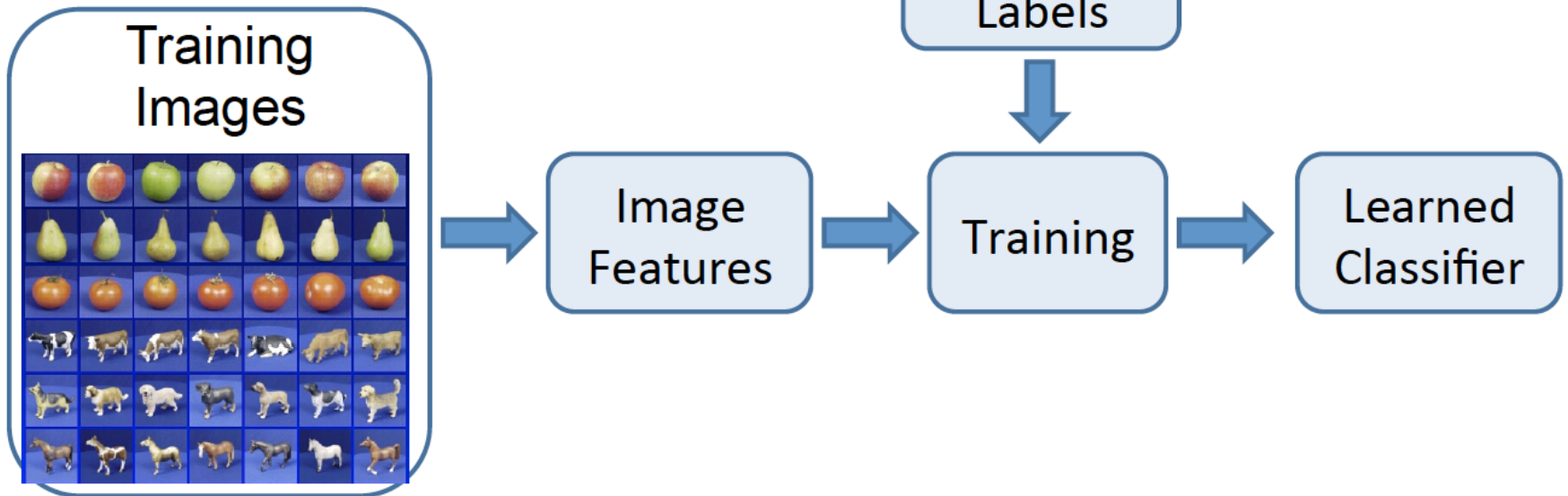
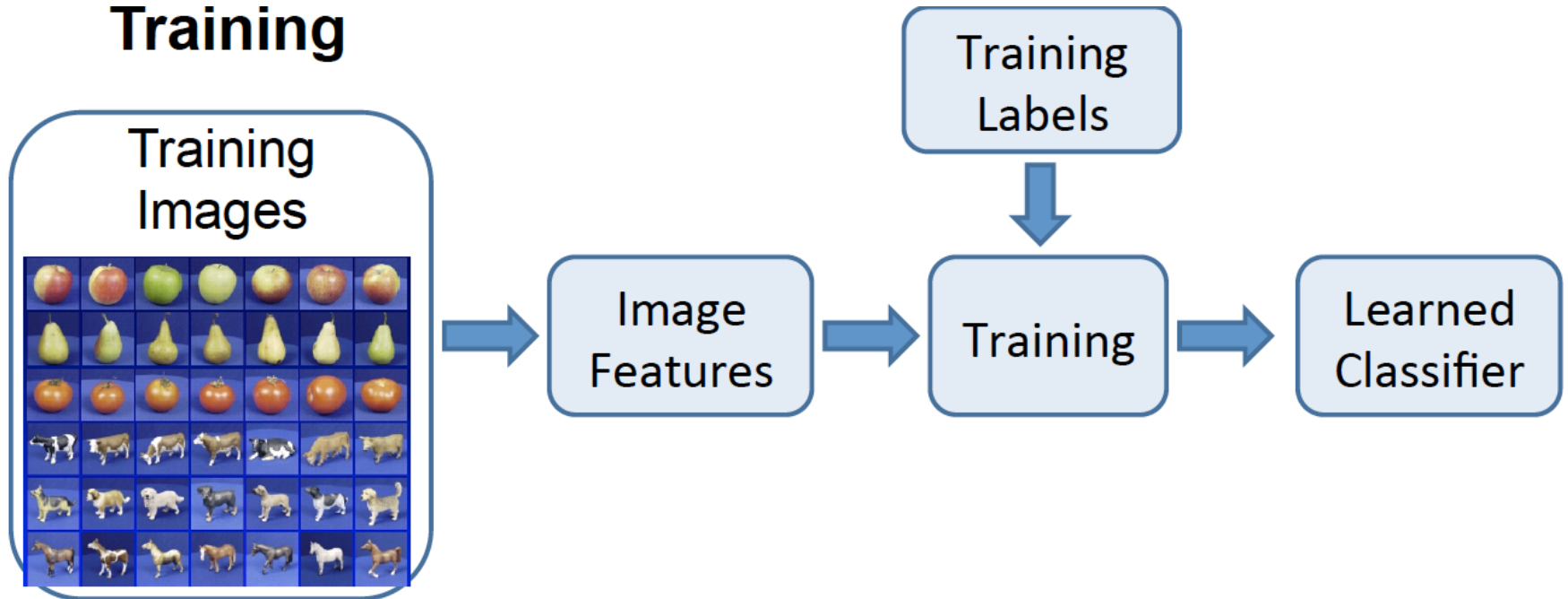


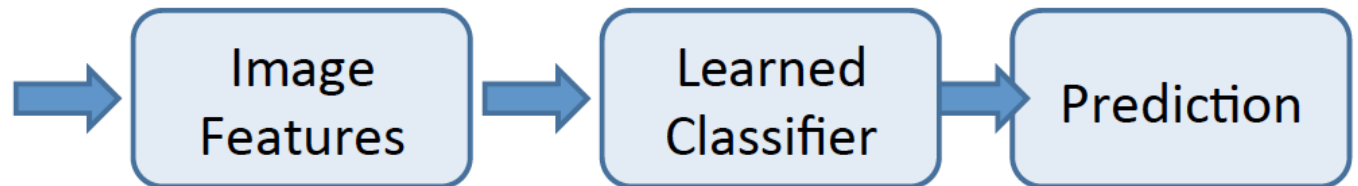
Image classification pipeline

Training



Testing

Test Image



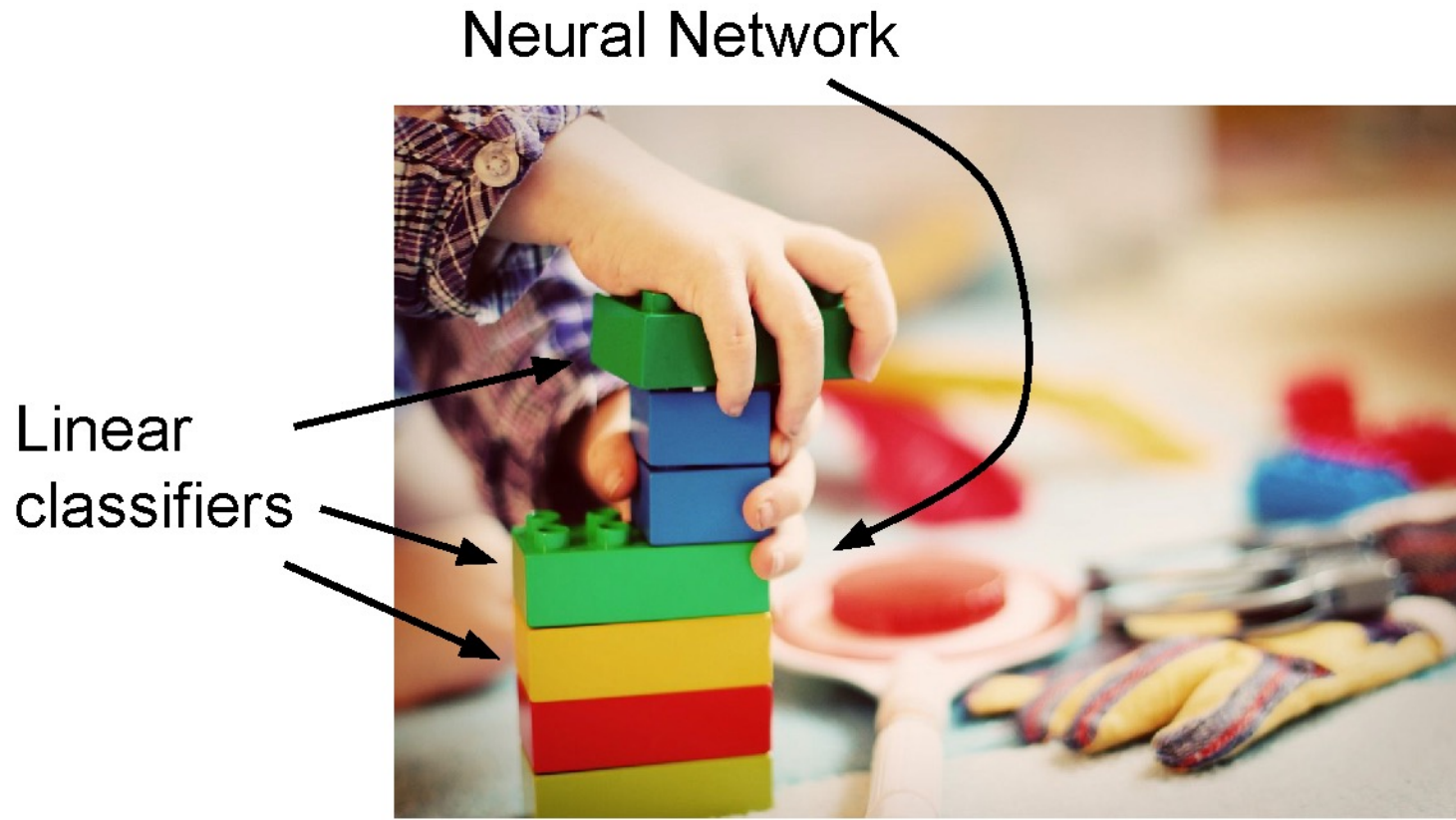
Summary

- What is machine learning
 - Learning a program (model) from data
- What is supervised learning
 - Supervision signal from labeled data
- How to solve a problem
 - Modeling: describe the problem by a mathematical model
 - Training: find the optimal model parameters (model fitting)
 - Testing: apply the trained model to new data
- Next: how to solve the classification problem?

Today

- Machine learning
- **Linear classifier**
- Neural networks
- Convolutional neural networks
- Training neural networks
- Network architectures

Why linear classifier important?



[This image](#) is [CC0 1.0](#) public domain

Classification model

- Input x : image
- Output y : class scores



class scores

Classification model

- What form should the function have?



image parameters

$f(\mathbf{x}, \mathbf{W})$

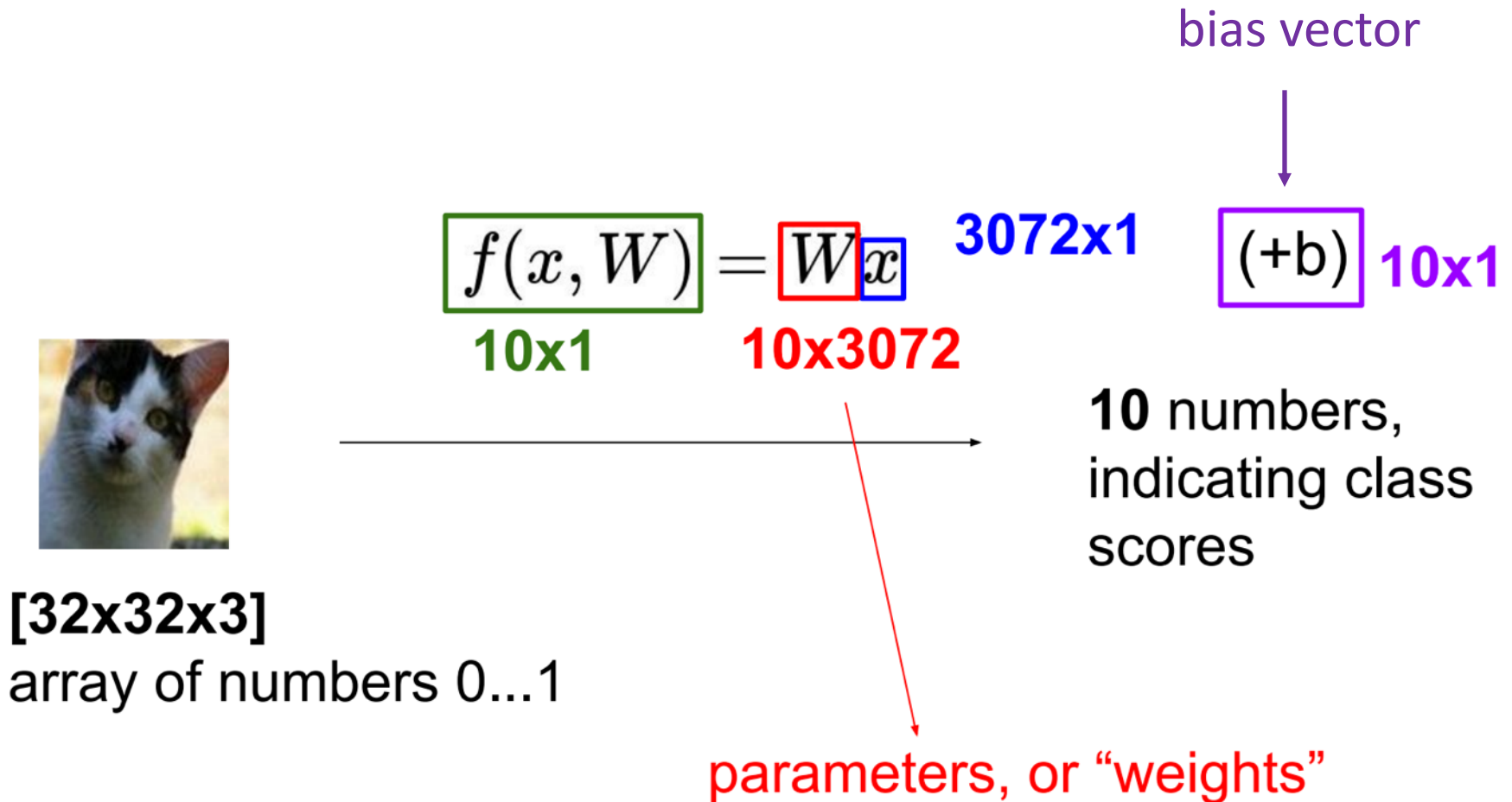


10 numbers,
indicating class
scores

[32x32x3]

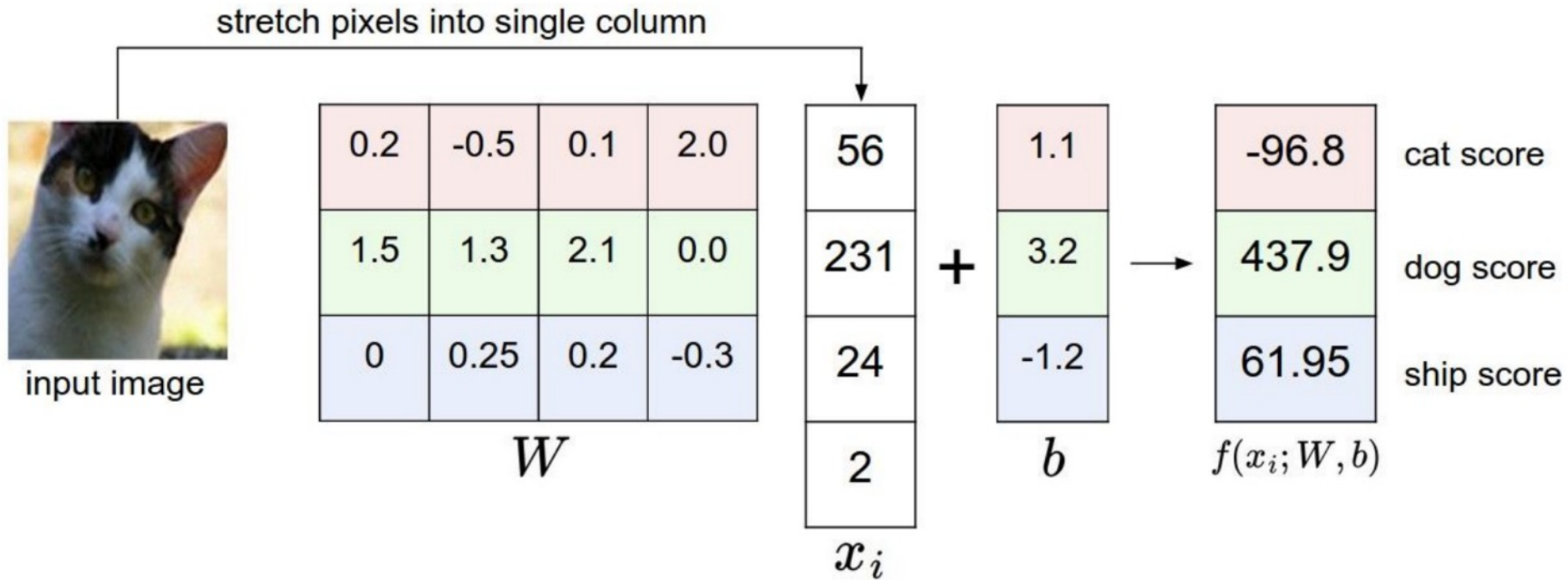
array of numbers 0...1
(3072 numbers total)

Linear classifier



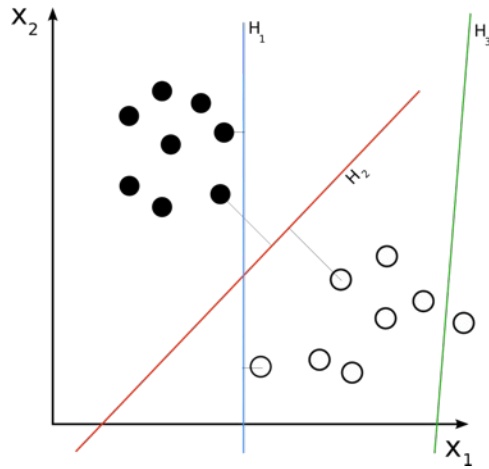
Linear classifier

Example with an image with 4 pixels, and 3 classes (cat/dog/ship)



Interpret linear classifier

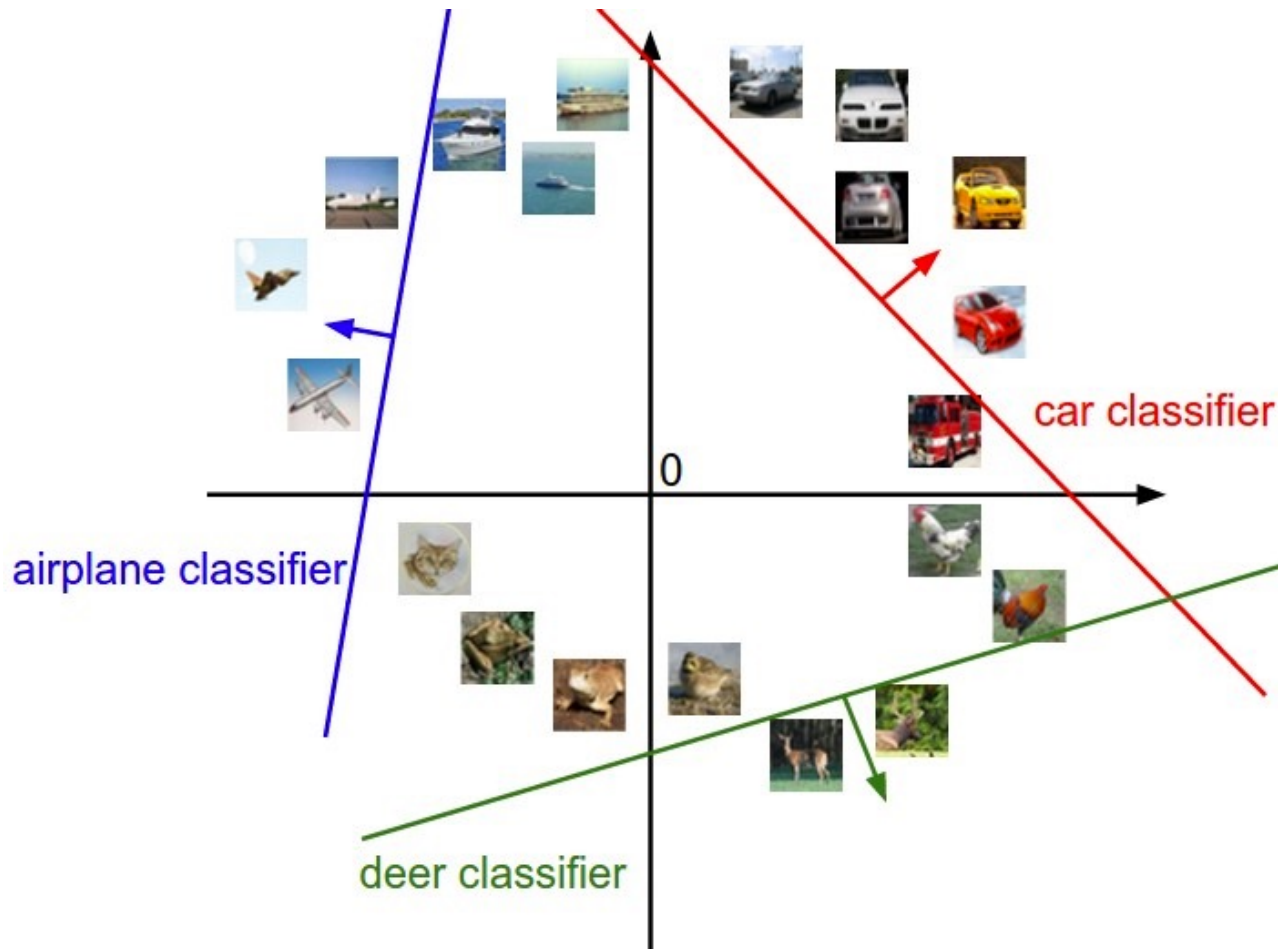
- When is the score $w^T x + b$ large?
 - When x (image) is similar to w (weights)
 - w represents how an object should look like
- We can decide the class label by thresholding the score, then $w^T x + b = 0$ is called decision boundary
 - A line in 2D, a plane in 3D and hyperplane in general



Each line H_i corresponds to a set of parameter (w, b)

Interpret linear classifier

- Multi-class



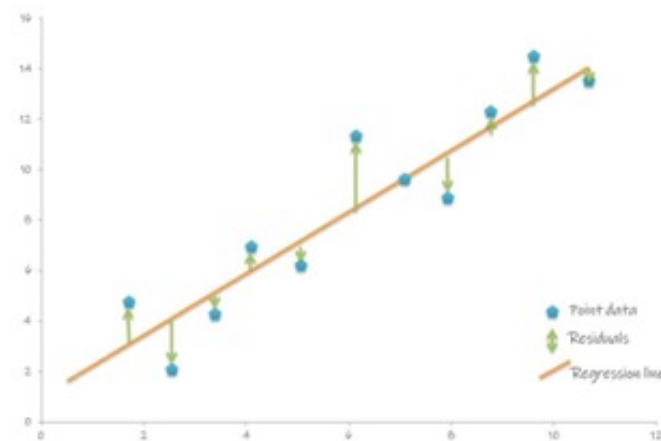
Training: how to find good parameters?

- Difficult to handcraft w
- Define a loss (objective) function that quantifies the “goodness” (likelihood) of w
- Solve the optimization problem

Loss function of classification

- Recap: loss function of regression:

$$\min_w \sum_i (y_i - f_w(x_i))^2$$



- Can we use the MSE loss for classification?

- No
- Labels are discrete and scores are real numbers



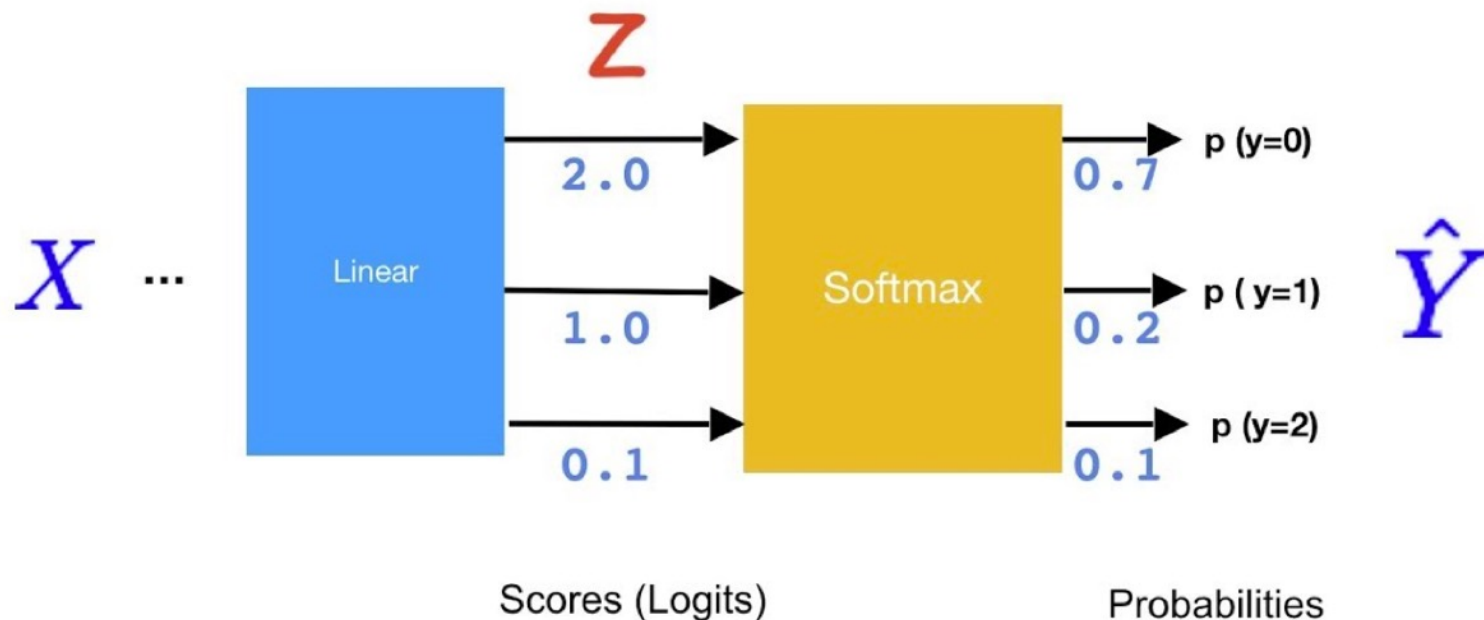
cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1

How to define a loss function

- Class labels can be viewed as probabilities
 1. Convert scores to probabilities
 2. Compute cross entropy between predicted and true probabilities

Convert scores to probabilities

Softmax operator: $\sigma(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}$ for $j = 1, \dots, K$.



Cross entropy as loss function

CROSS-ENTROPY

$S(Y)$

0.7
0.2
0.1

L

1.0
0.0
0.0

$$D(S, L) = - \sum_i L_i \log(S_i)$$

Summary

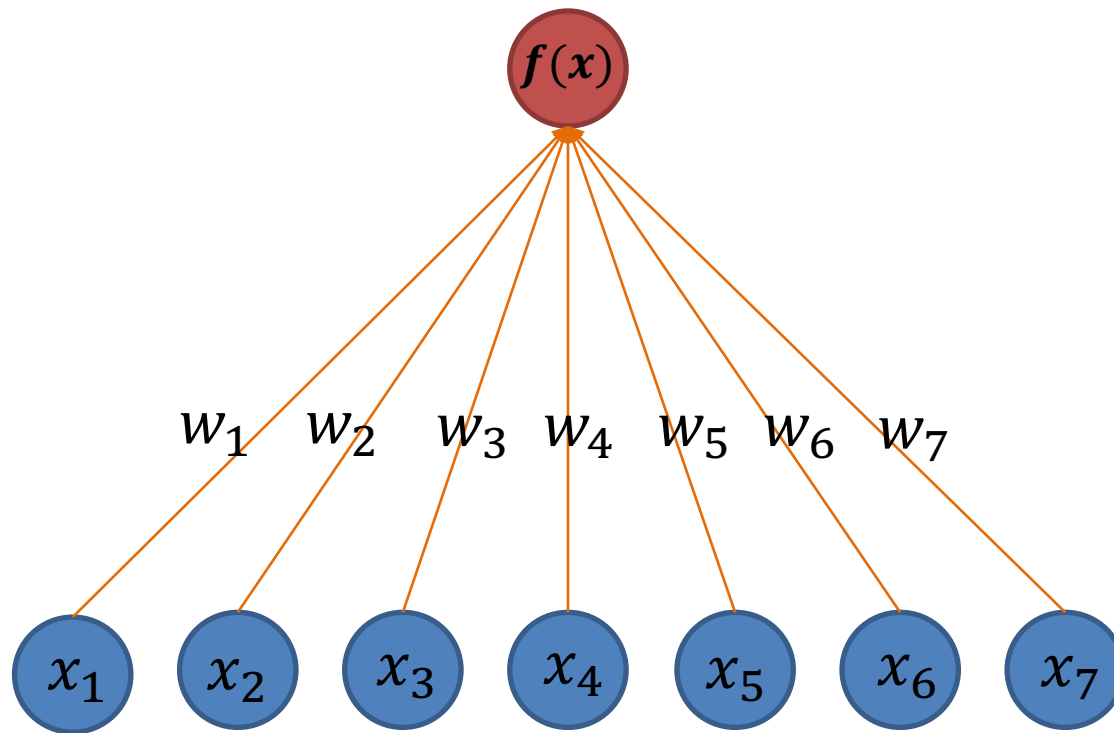
- Linear classifier
- Softmax function
- Cross-entropy loss

Today

- Machine learning
- Linear classifier
- **Neural networks**
- Convolutional neural networks
- Training neural networks
- Network architectures

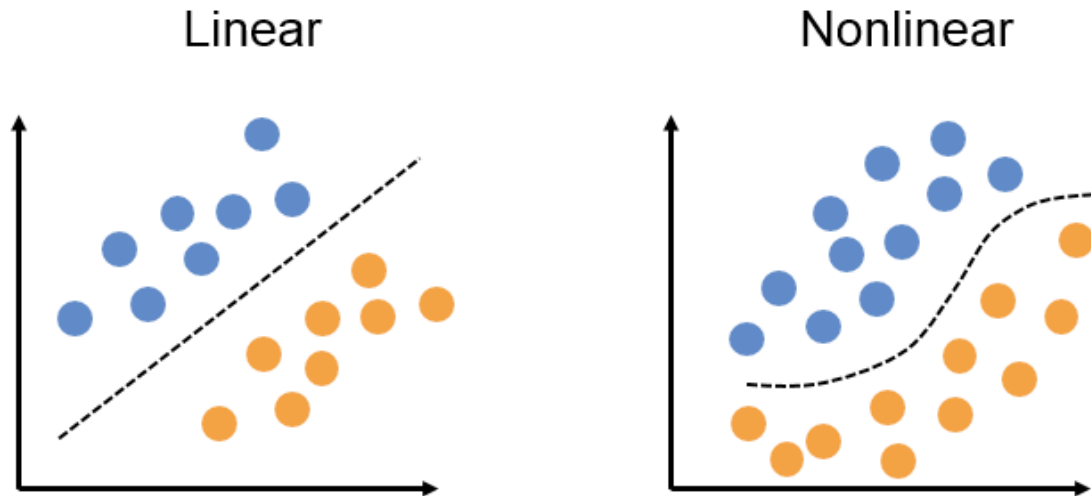
Graphical representation

- Perceptron



Limitation of linear model

- The data may be NOT linear separable



<https://jtsulliv.github.io/perceptron/>

How to model nonlinearity

- Adding a nonlinear transform

$$f(x) = \sigma(w^T x + b)$$

- Named activation functions

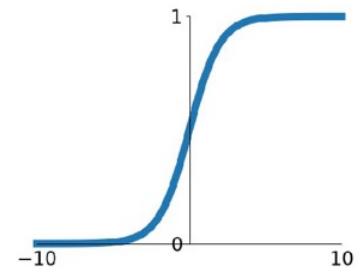
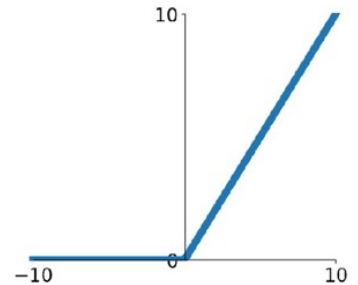
- Examples:

- Rectified Linear Unit (ReLU): $\sigma(z) = \max(0, z)$

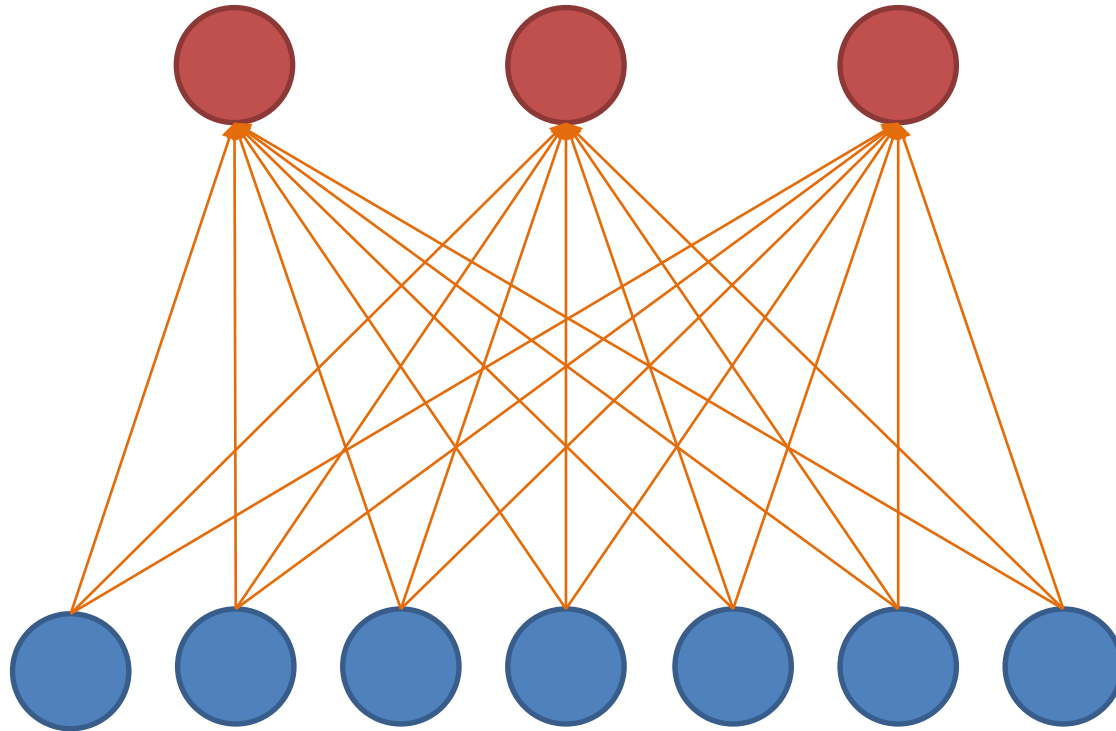
- “Electrical pulse” cannot be negative

- Sigmoid: $\sigma(z) = 1/(1 + e^{-z})$

- Map values to probability



How to represent multiple outputs



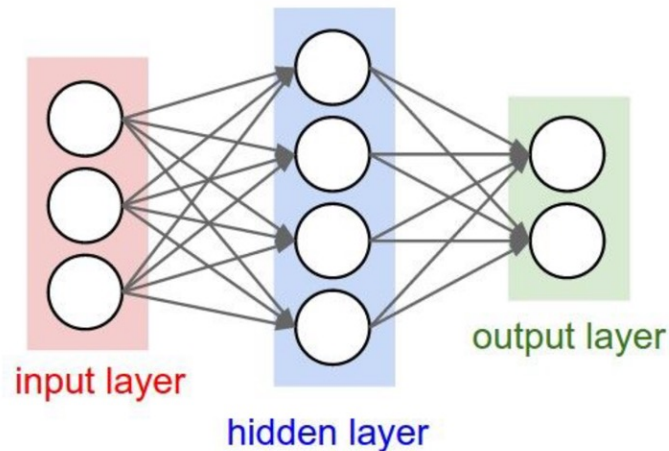
$$f(x) = \sigma(w^T x + b) \longrightarrow f(x) = \sigma(Wx + b)$$

↑
matrix

Multi-layer perceptron

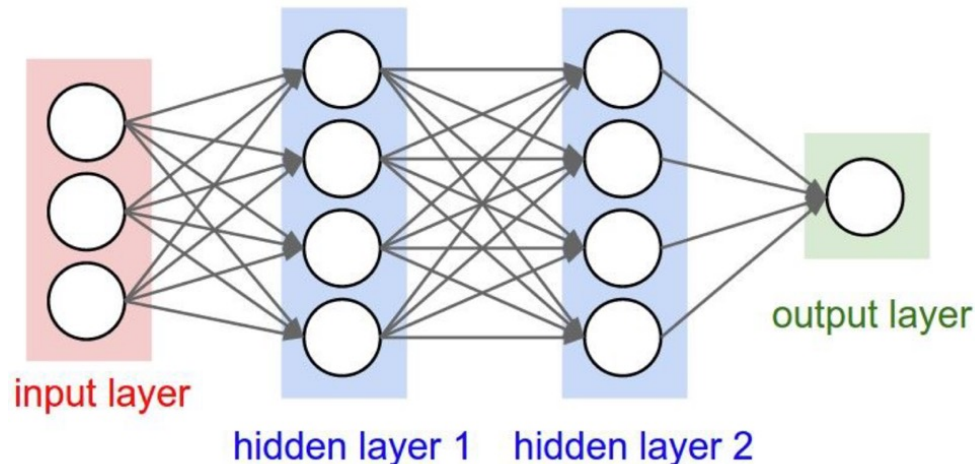
- $f(x) = \sigma(Wx + b)$ might be too simple to represent a complicated decision boundary
- We can it to composite a more complicated function:

$$f(x) = \sigma(W_2\sigma(W_1x + b_1) + b_2)$$



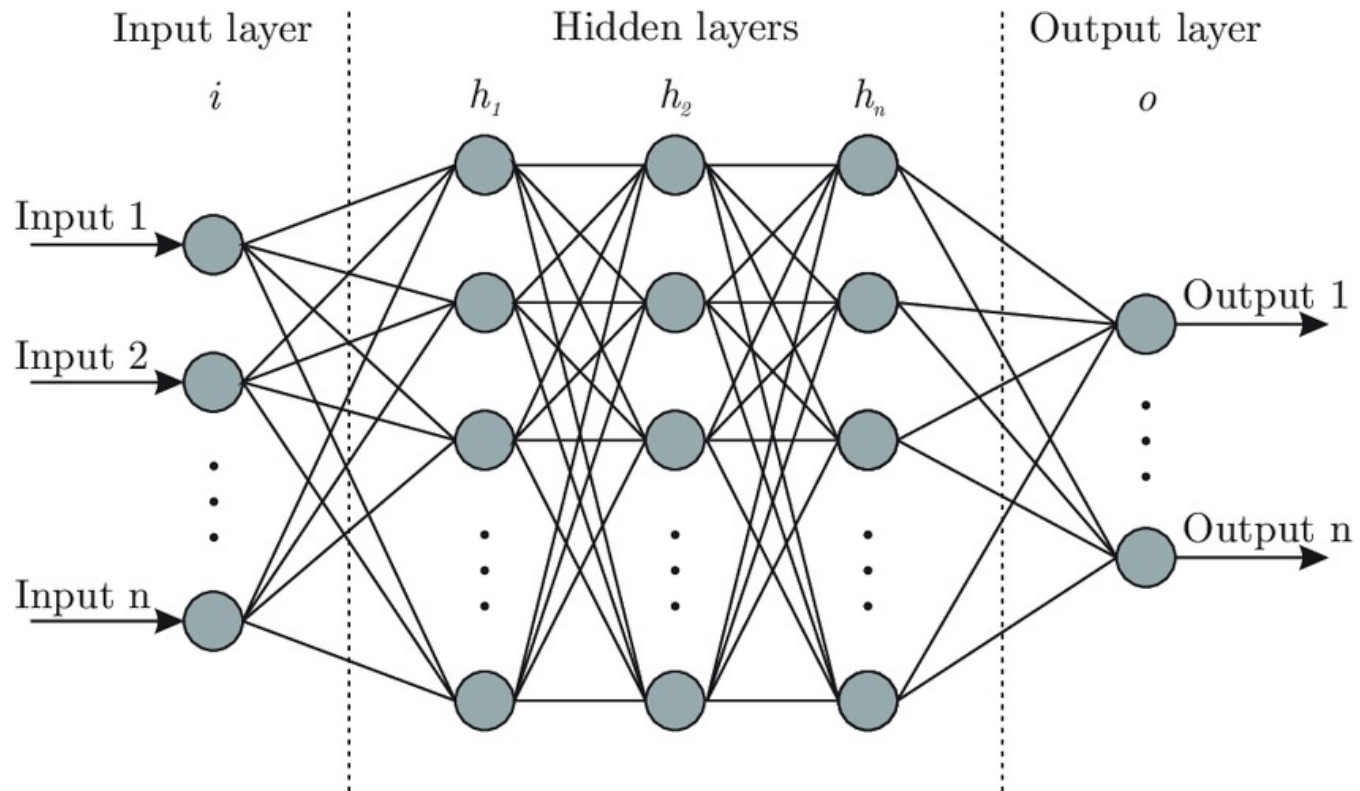
Multi-layer perceptron

- $f(x) = \sigma(Wx + b)$ might be too simple to represent a complicated decision boundary
- We can use it to composite a more complicated function:
$$f(x) = \sigma(W_3 \sigma(W_2 \sigma(W_1 x + b_1) + b_2) + b_3)$$



What will happen if there is no activation functions?

Neural networks

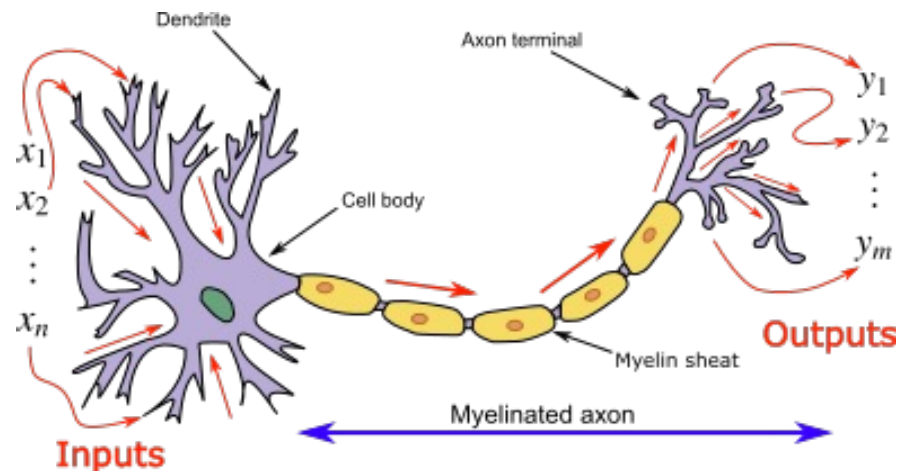


Neural networks

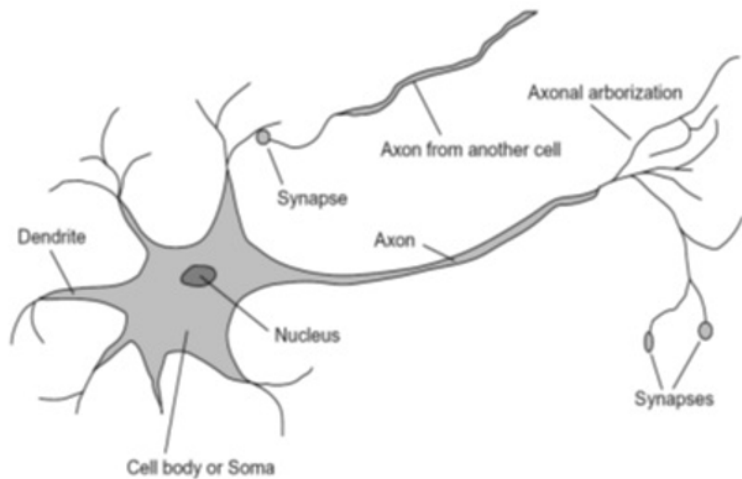
- (Artificial) neural networks – linear functions chained together and separated by non-linear activation functions
- Used to regress the function that maps input to output
- A computational model inspired by human nervous system

Information processing in human brain

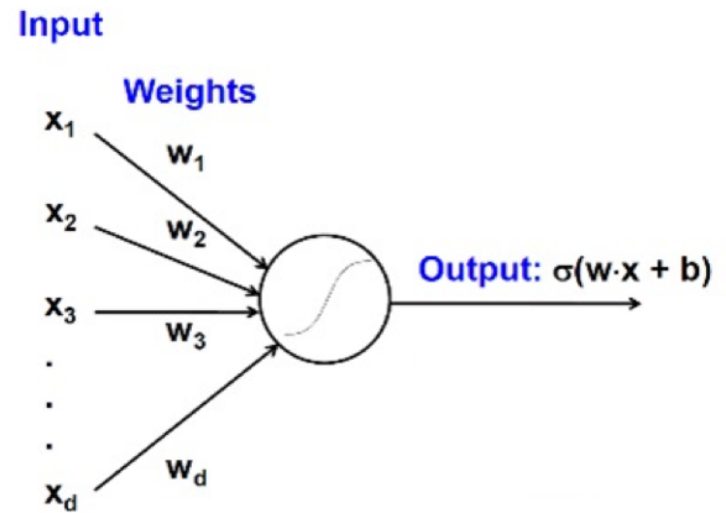
- Very mysterious
- Some facts:
 - Nervous system is composed of many many neurons.
 - A neuron is a nerve cell that carries electrical impulses.



Artificial neuron



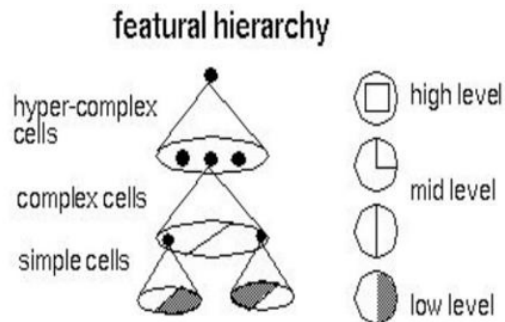
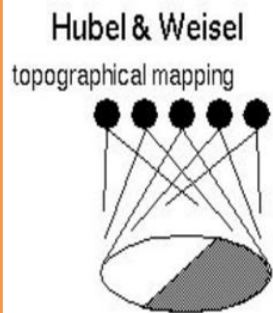
A biological neuron



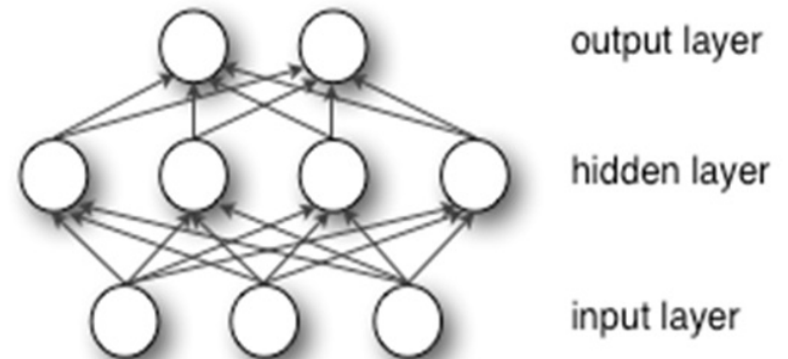
An artificial neuron



Artificial neural networks



Hubel and Weisel's architecture



Artificial Neural Network



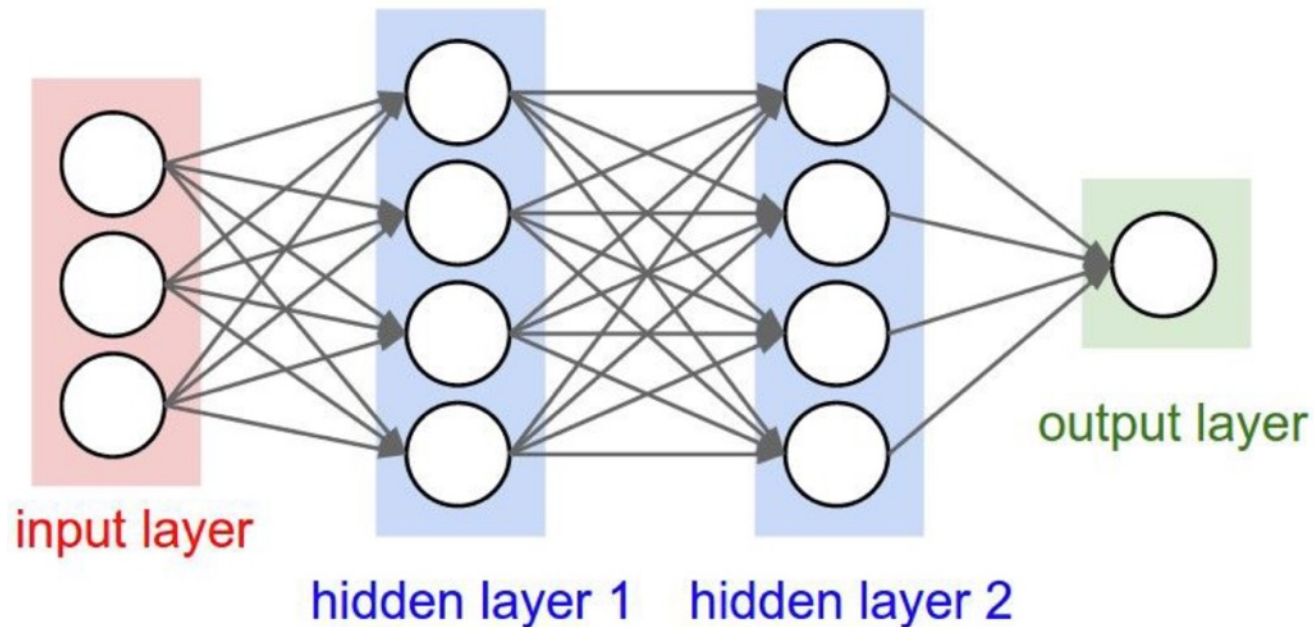
Deep Neural networks

- Deep neural networks
 - neural networks with many layers
- A network with more layers mean is able to represent more complex function
- The more the better?
 - More parameters to learn
 - need more data
 - More difficult to train
 - “gradient vanishing”



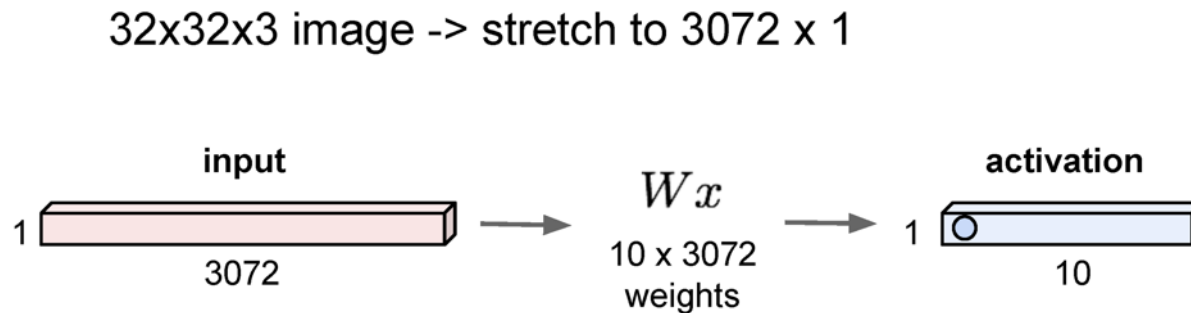
Fully connected layer

- Nodes are fully connected to the nodes in the previous layer



Fully connected layer

- Nodes are fully connected to the nodes in the previous layer



Fully connected layer

- Fully connected layer has a large number of weights

$$f(x) = \sigma(Wx + b)$$

- Deep networks typically have many layers and potentially millions of parameters
- What problem will happen if there are too many parameters?
- How to reduce number of parameters?

Summary

- Perceptron
- Activation function
- Neural networks – multi-layer perceptron
- Mathematical expression of neural networks
- Fully connected layer

Today

- Machine learning
- Linear classifier
- Neural networks
- **Convolutional neural networks**
- Training neural networks
- Network architectures

Convolutional neural networks

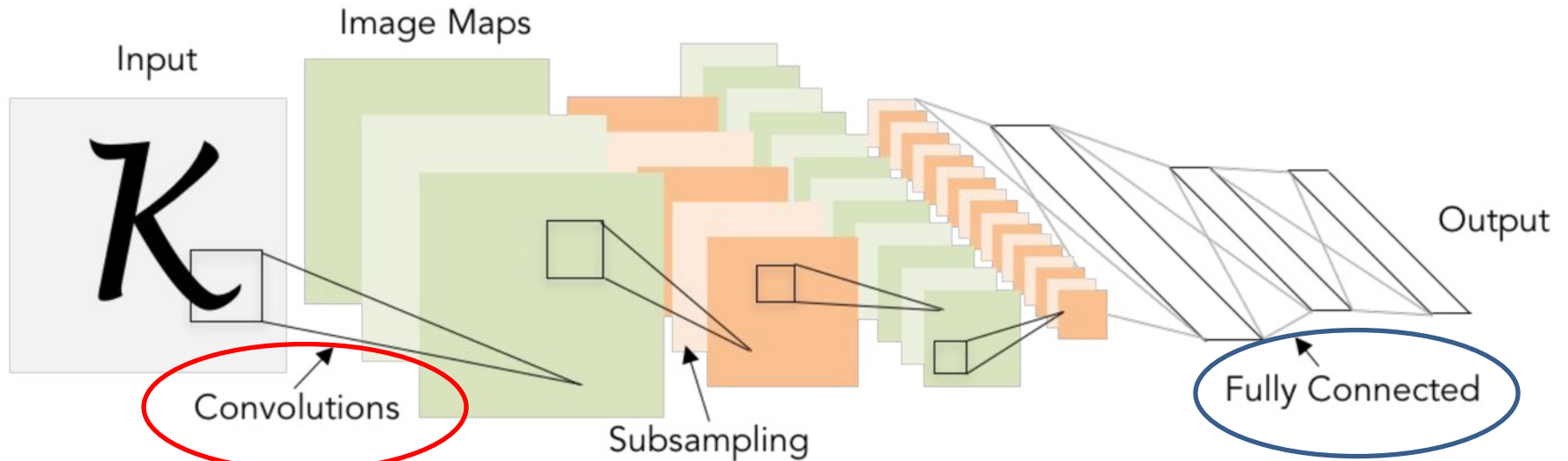


Illustration of LeCun et al. 1998 from CS321, 2017 Lecture 1

Convolutional neural networks

- We can recognize an image from local patterns



Locally connected network

- Looking at local patterns

FULLY CONNECTED NEURAL NET

Example: 1000x1000 image
1M hidden units

→ 10^{12} parameters!!!

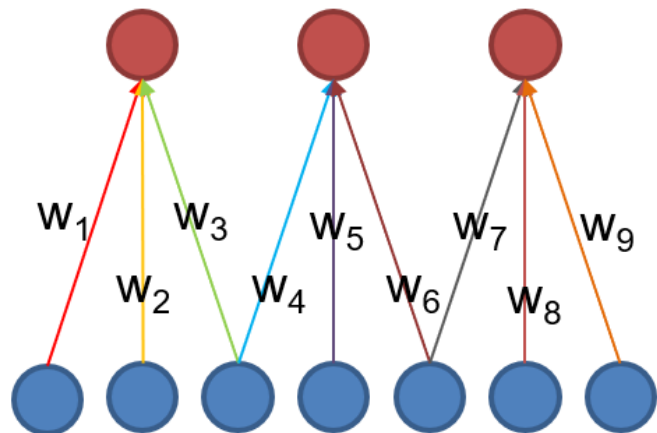
- Spatial correlation is local
- Better to put resources elsewhere!

LOCALLY CONNECTED NEURAL NET

Example: 1000x1000 image
1M hidden units
Filter size: 10x10
100M parameters

Locally connected network

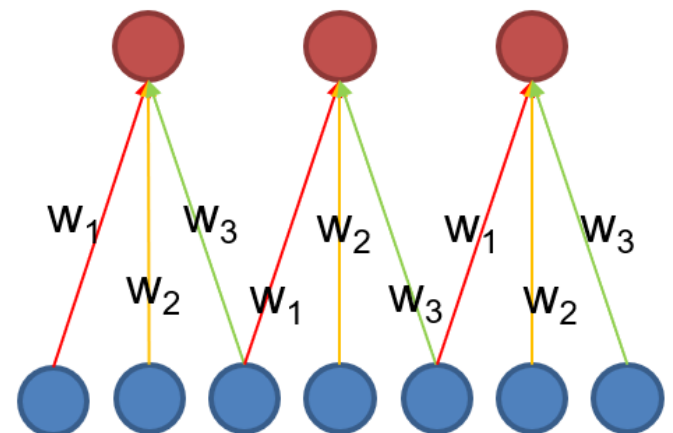
- Number of parameters
 - Global connectivity: 7
 - Local connectivity: $3 \times 3 = 9$



Hidden layer

Input layer

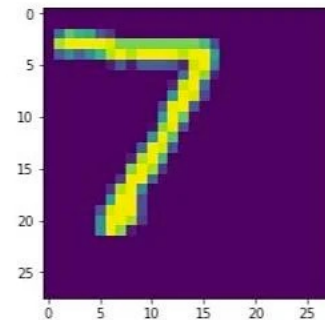
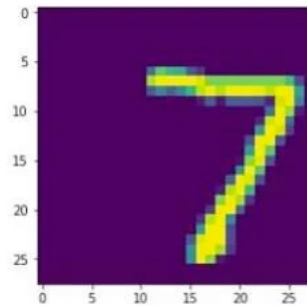
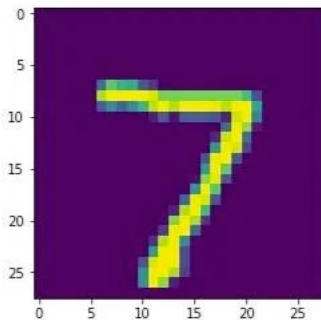
Without weight sharing



With weight sharing

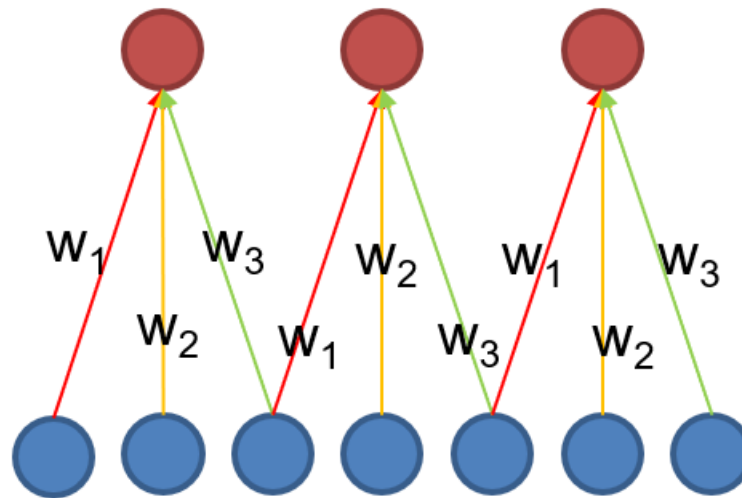
Weight sharing

- Why is weight sharing a valid assumption?
 - Reminder: weights represent how an object should look like
 - Should not change with its position in the image (Shift invariance property)



Convolution

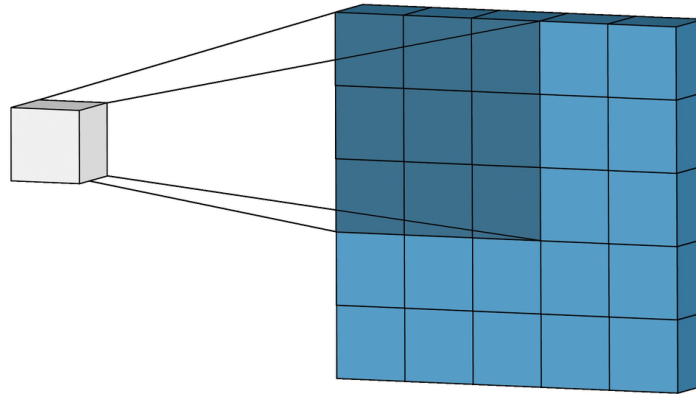
Local connectivity + weight sharing = convolution!



$$y = \sigma(x \otimes w + b)$$

Convolution

- 2D convolution



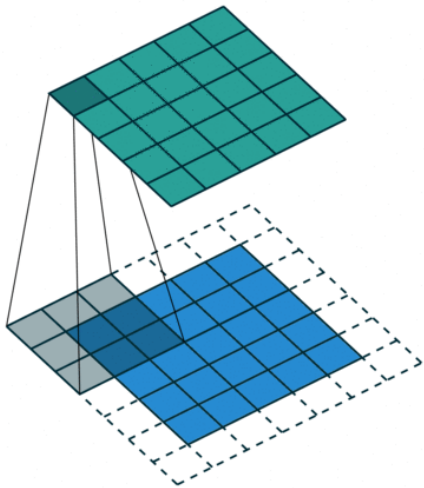
Q: what is the size of output?

A: $\text{ImageSize} - \text{FilterSize} + 1$

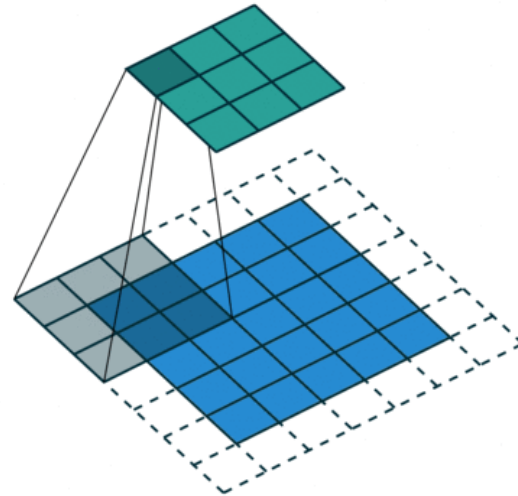
Convolution

- Padding and stride

Padding: adding zero pixels around the border



Stride: step size larger than 1 in convolution

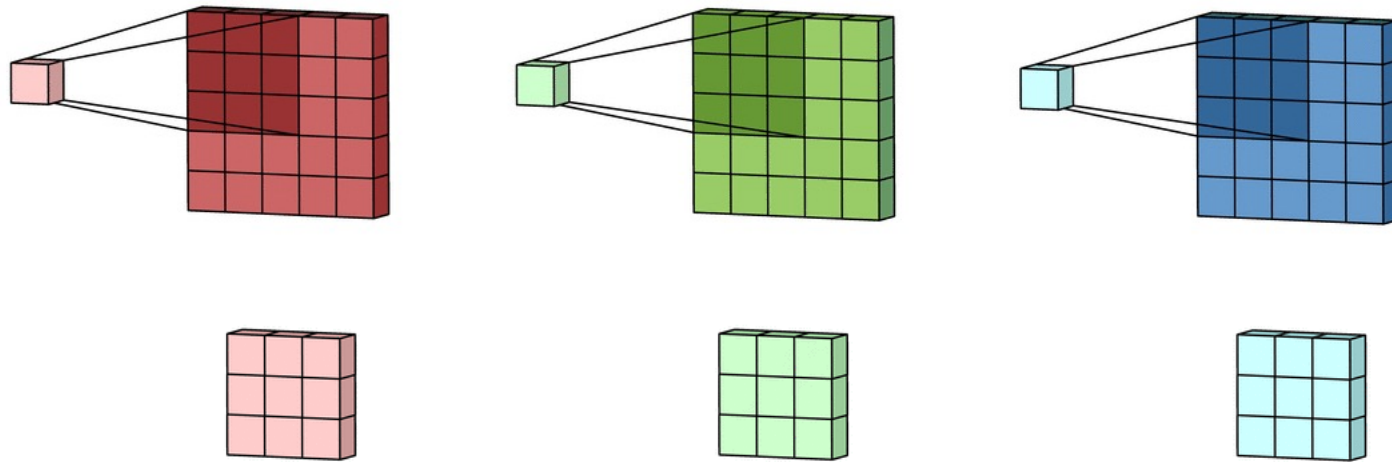


Q: what is the size of output?

A: $(\text{ImageSize} - \text{FilterSize} + \text{Padding} * 2) / \text{Stride} + 1$

Convolution

- What if the input has a 3rd dimension (e.g. RGB)?

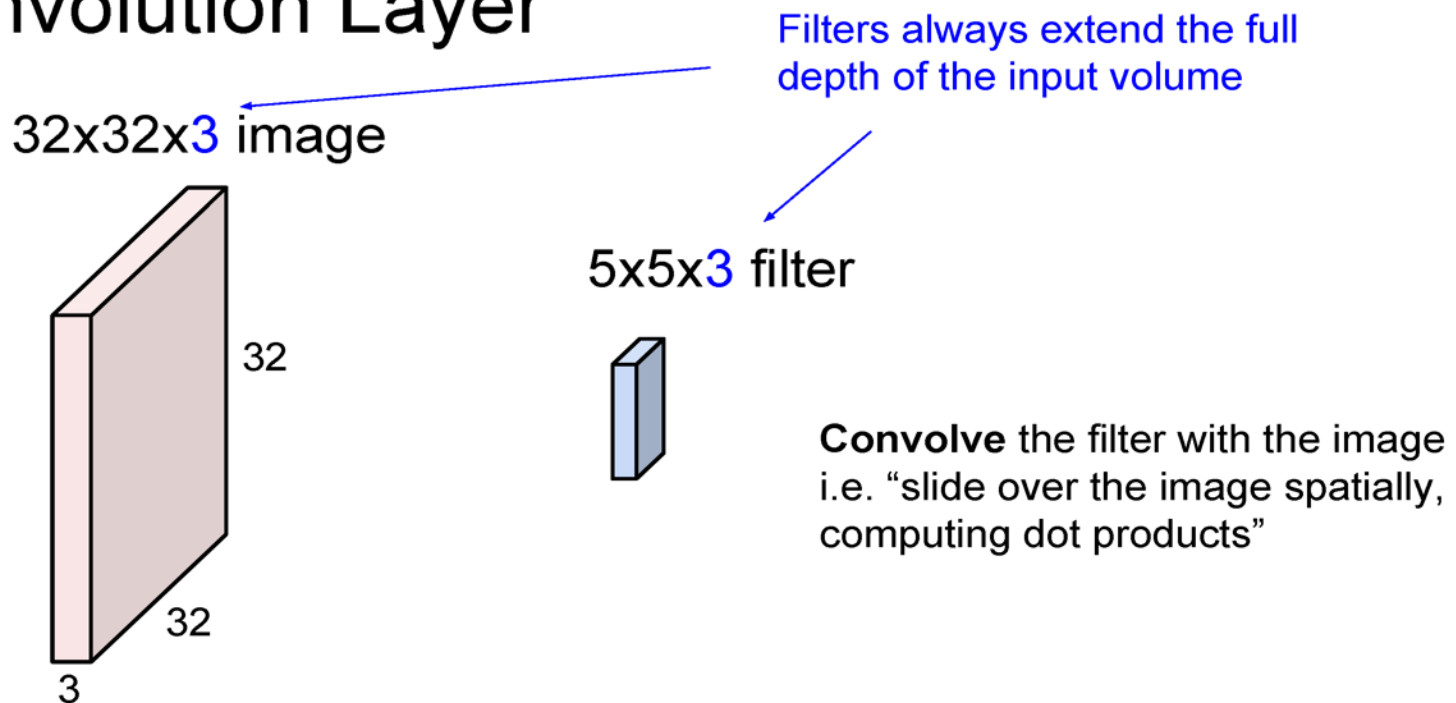


Output is still single-channel!

Convolution

- Convolution of multi-channel image(3D tensor)

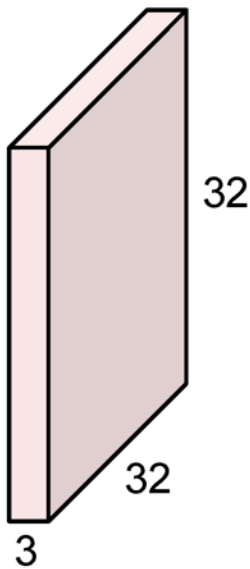
Convolution Layer



Convolution

- Much fewer parameters than fully connected layer

32x32x3 image



5x5x3 filter

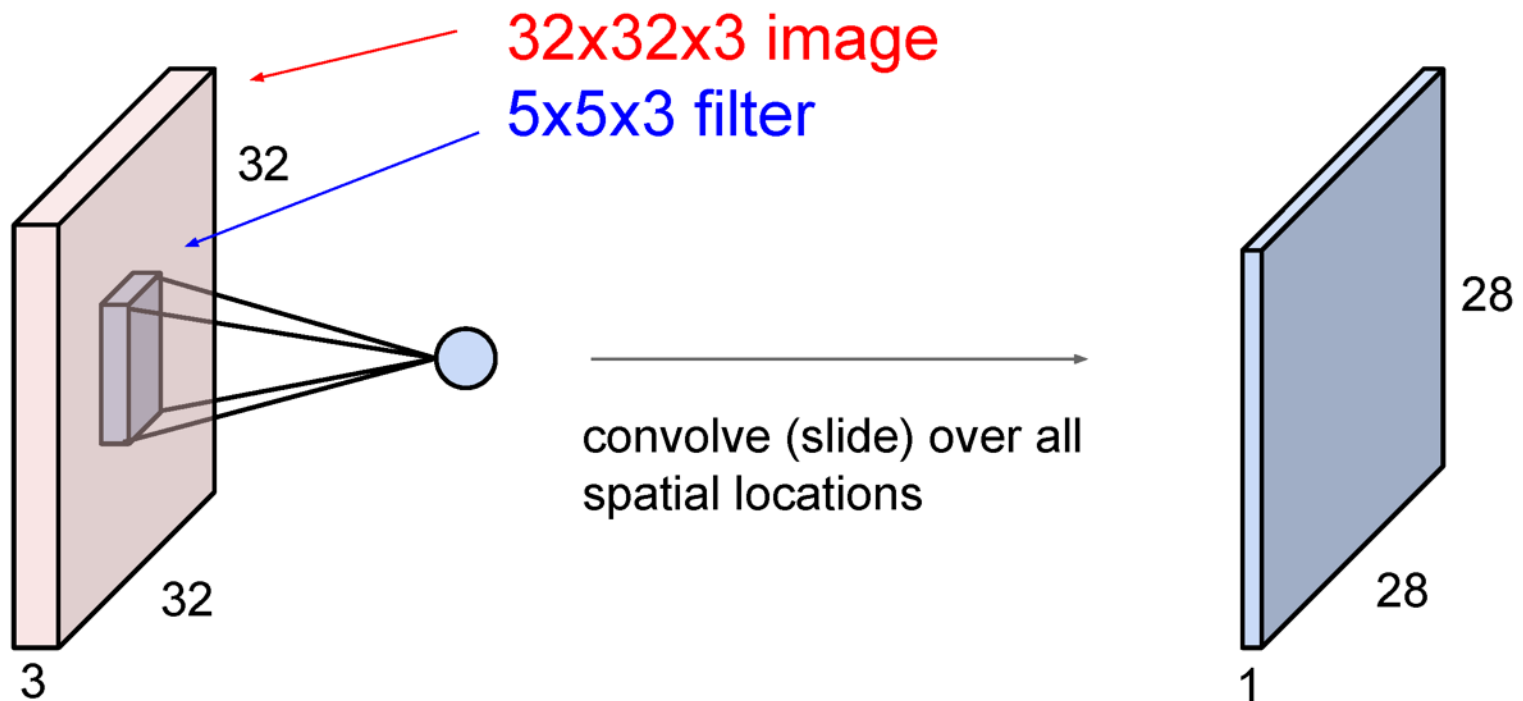


Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

Number of weights: $5 \times 5 \times 3 + 1 = \mathbf{76}$
(vs. 3072 for a fully-connected layer)

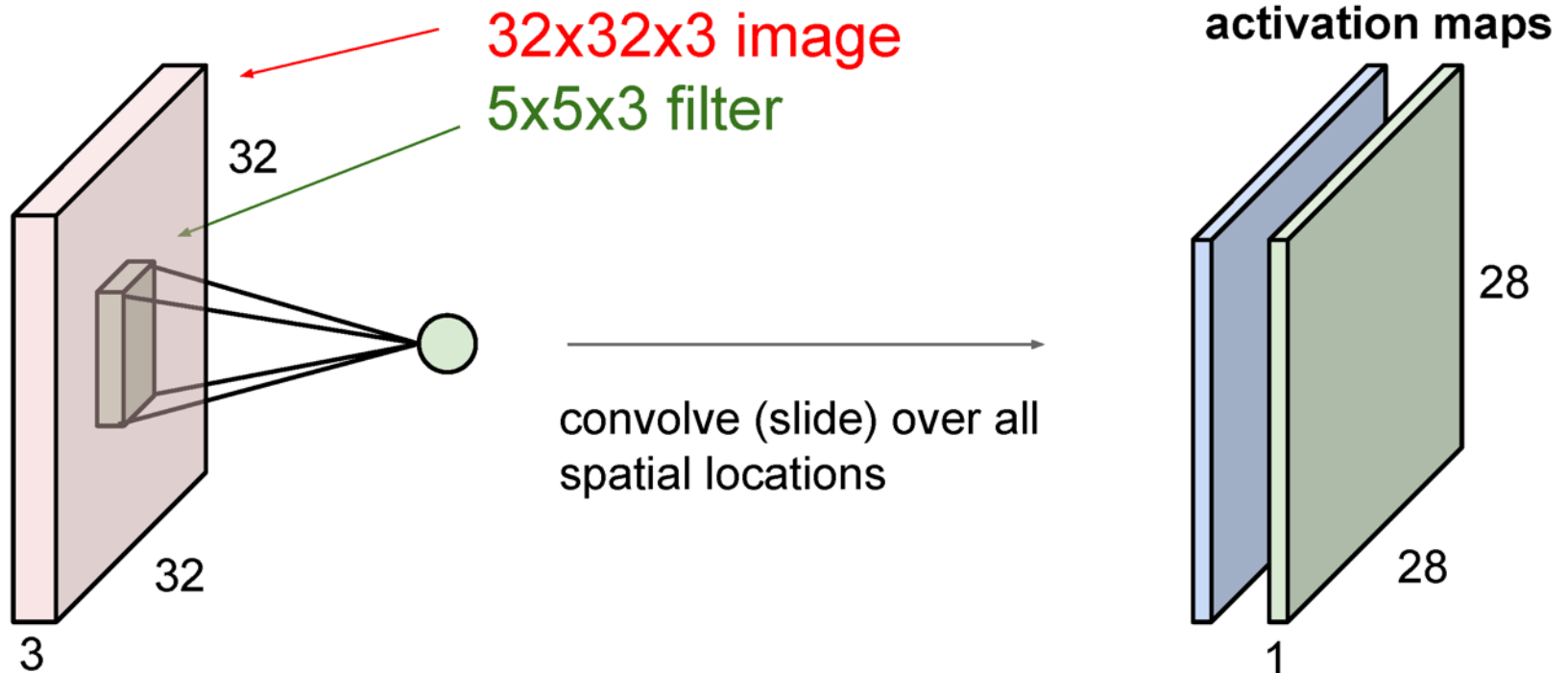
Convolution layer

- Input: image
- Output: feature map (activation map)
- Parameters: filter



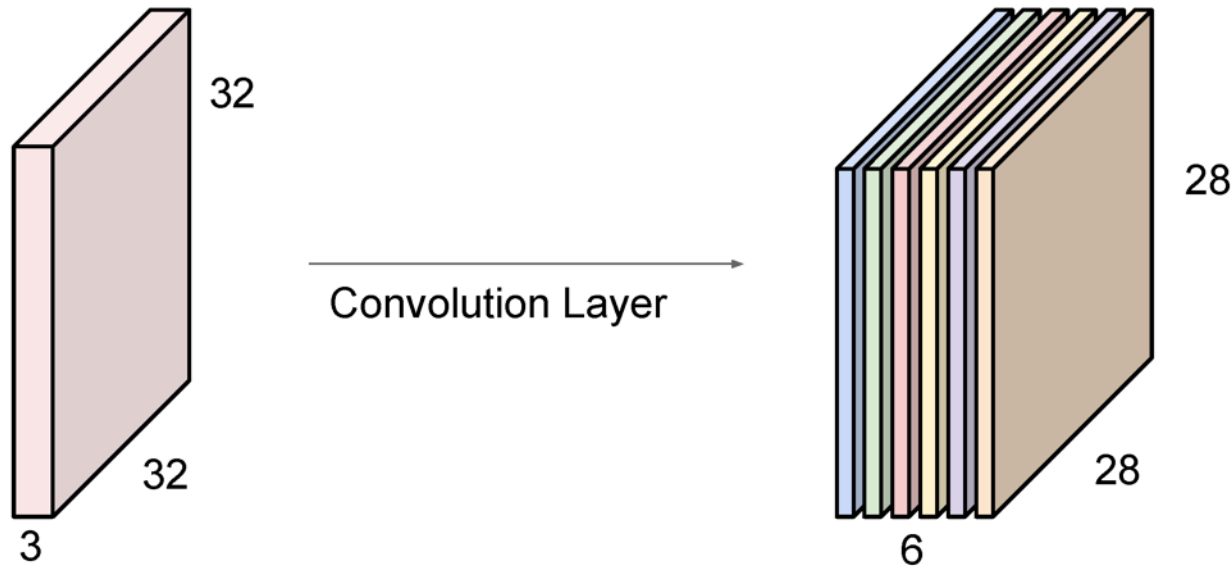
Convolution layer

- Could be more than 1 filter
- The output are multiple feature maps



Convolution layer

- Number of filters = number of feature maps

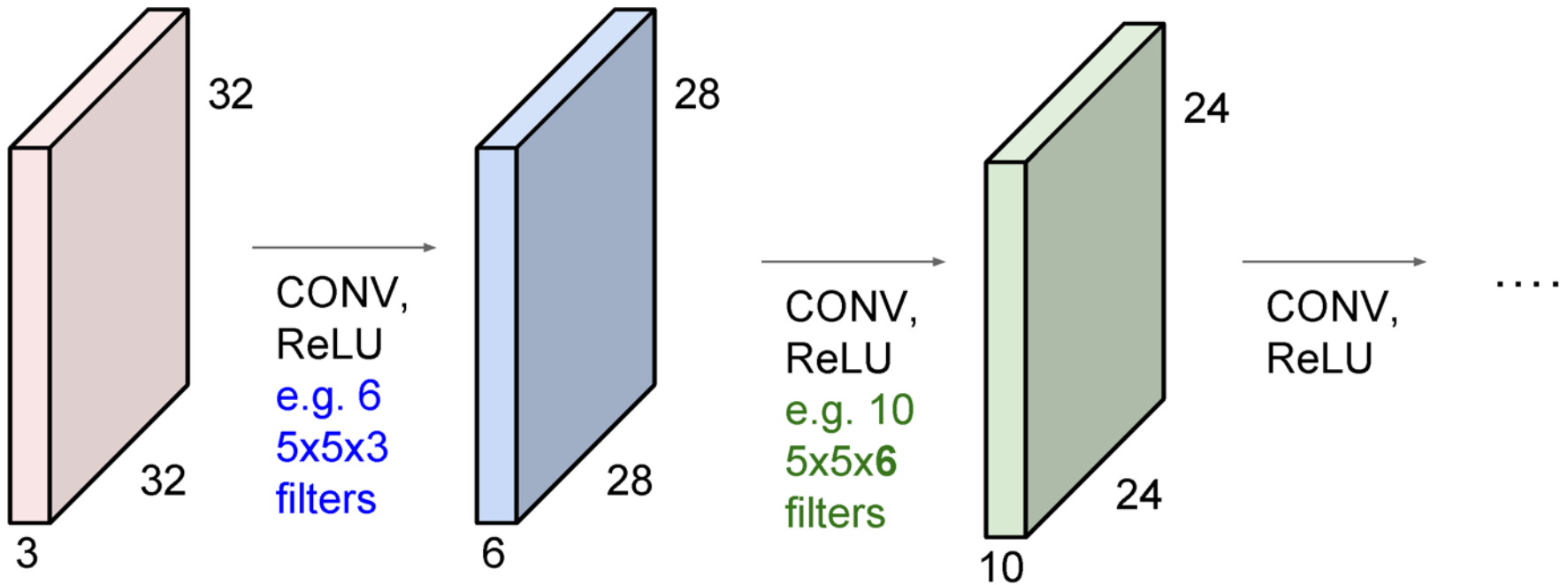


We stack these up to get a “new image” of size 28x28x6!

(total number of parameters: $6 \times (75 + 1) = \mathbf{456}$)

Convolution neural network(CNN)

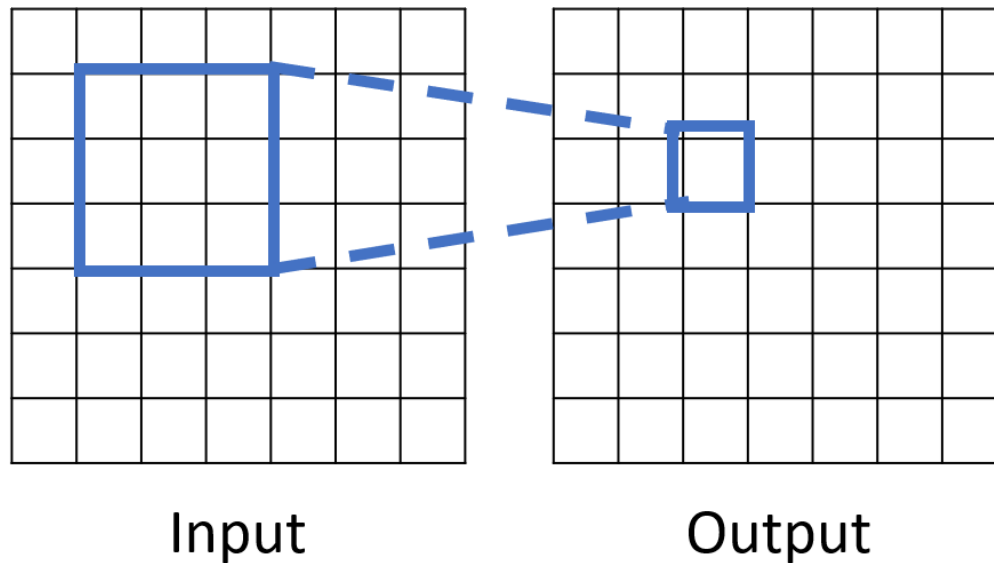
- CNN is a sequence of convolution layers separated by activation functions



Reminder: neural network is a sequence of linear functions separated by activation functions

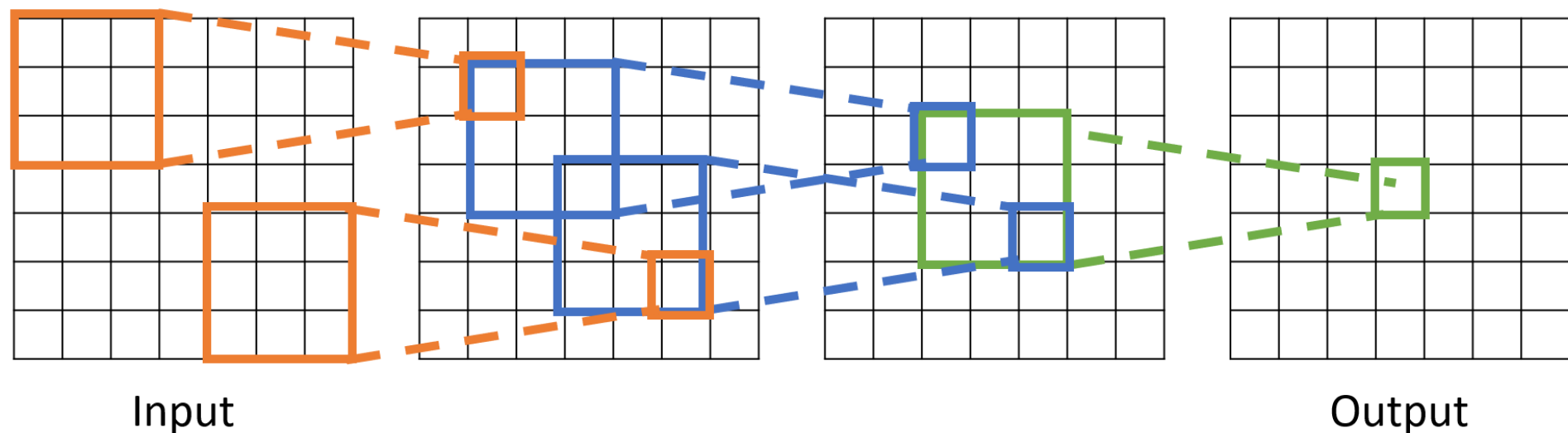
Receptive fields

- A convolution layer with kernel size K , each element in the output depends on $K \times K$ receptive field in the input



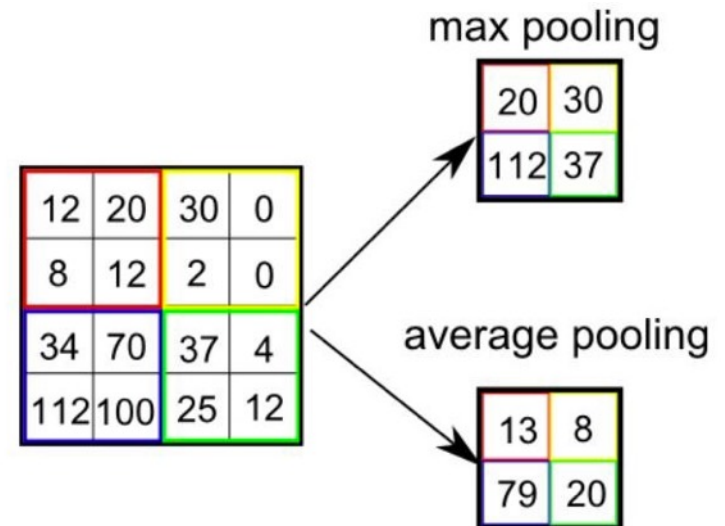
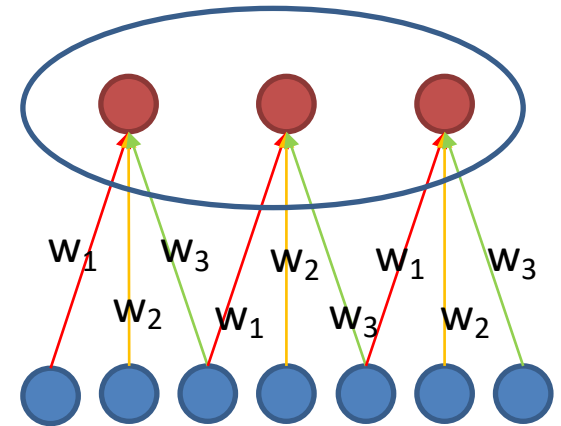
Receptive fields

- For successive convolution with K kernel size and L layers, the receptive field is $1 + L * (K - 1)$

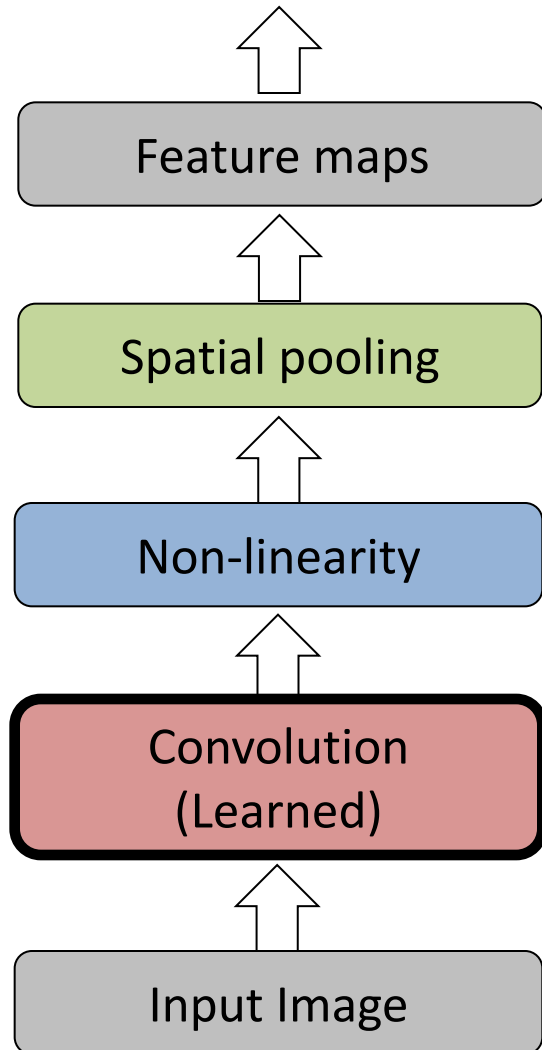


Pooling layer

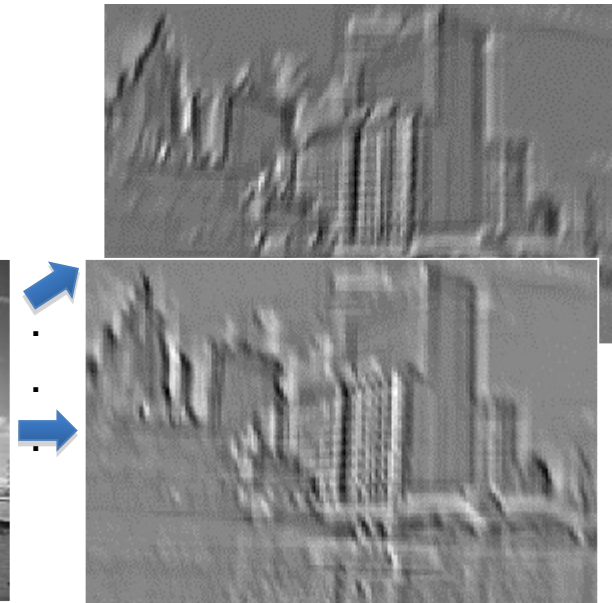
- Pooling: combine outputs of a filter at different locations
- Pooling operators
 - Max pooling
 - Average pooling
- Why pooling?
 - Aggregate spatial information
 - Makes feature maps smaller and more manageable



Convolution neural network(CNN)

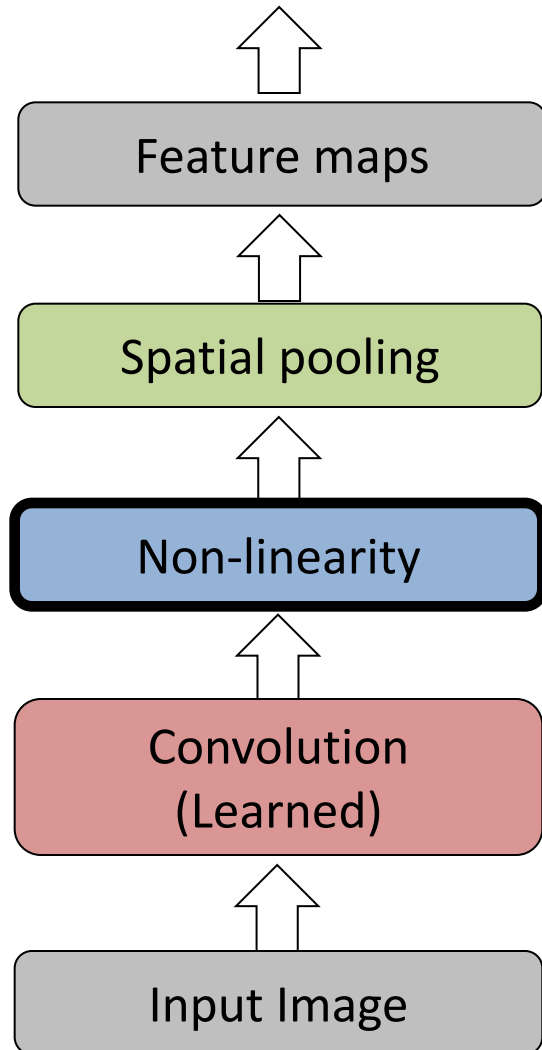


Input

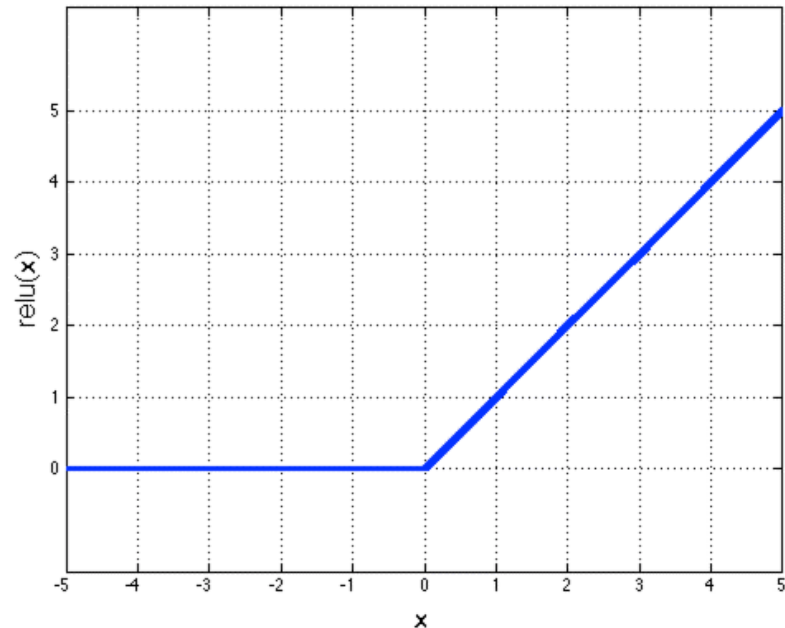


Feature Map

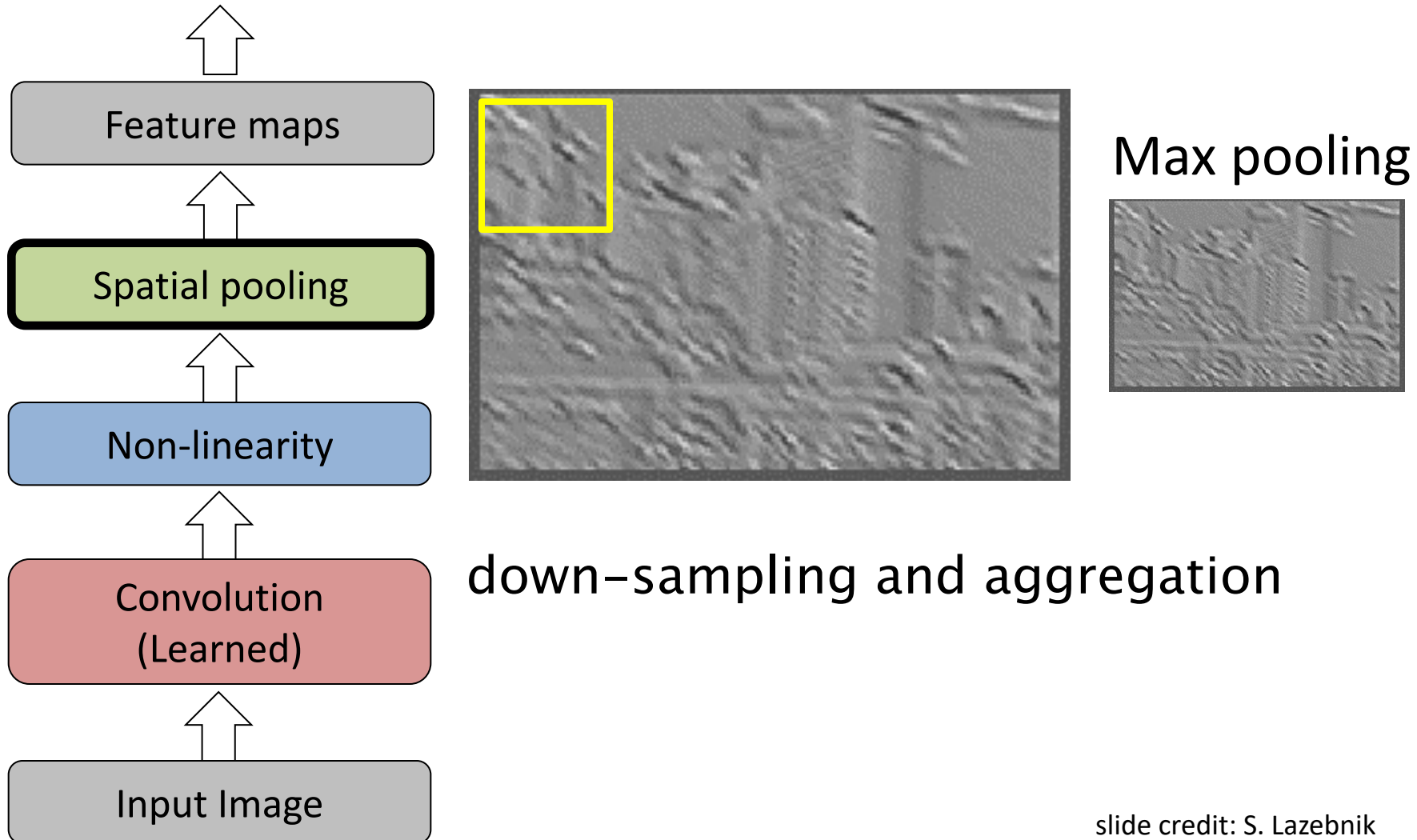
Convolution neural network(CNN)



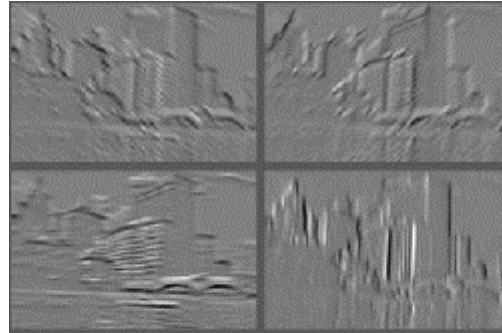
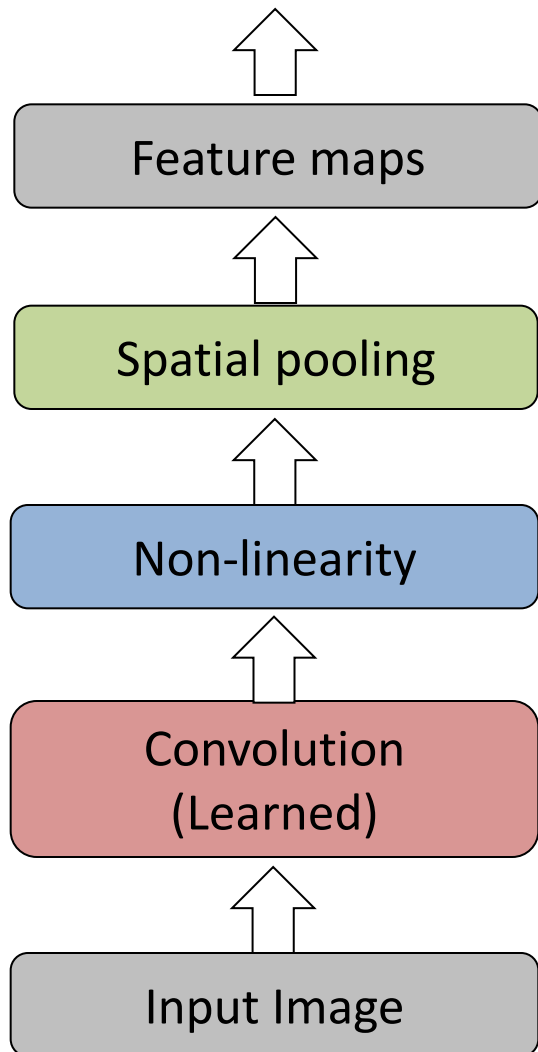
Rectified Linear Unit (ReLU)



Convolution neural network(CNN)



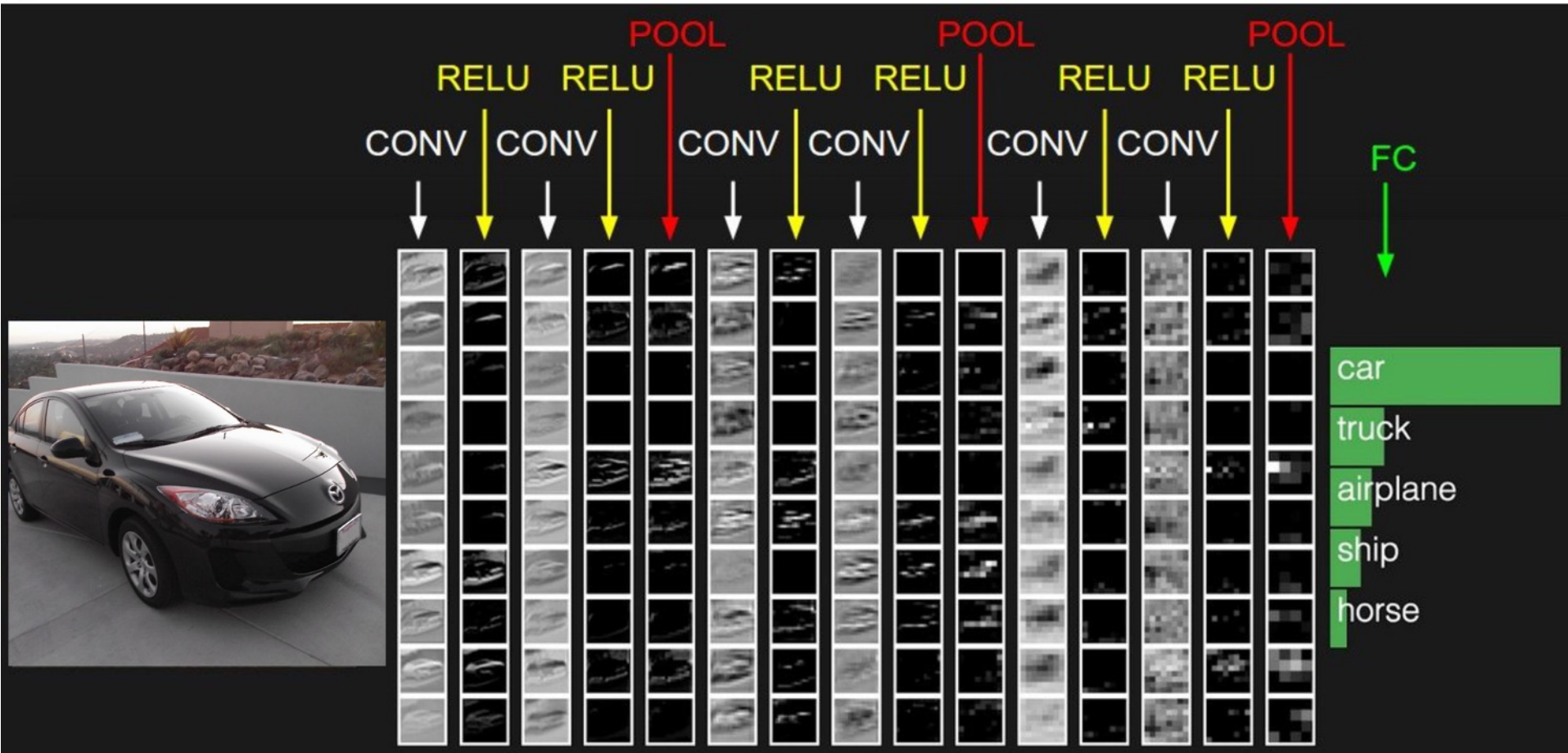
Convolution neural network(CNN)



Feature Maps

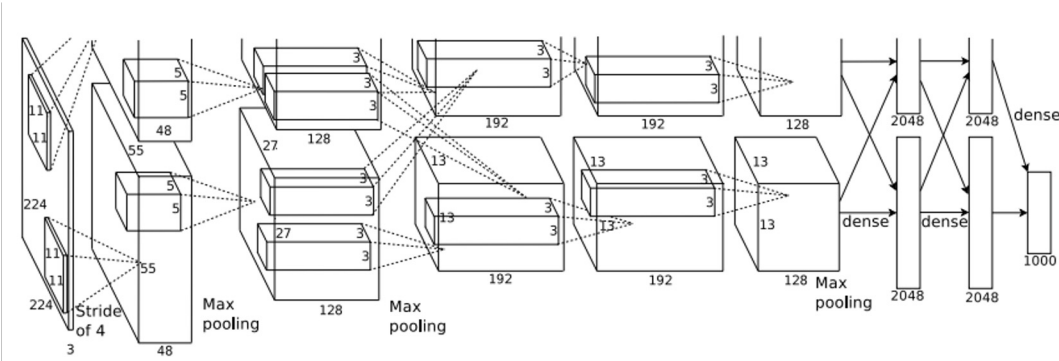
CNN for classification

preview:



The final fully connected layer converts the feature maps into class scores

AlexNet



Full (simplified) AlexNet architecture:

[227x227x3] INPUT

[55x55x96] **CONV1**: 96 11x11 filters at stride 4, pad 0

[27x27x96] **MAX POOL1**: 3x3 filters at stride 2

[27x27x96] **NORM1**: Normalization layer

[27x27x256] **CONV2**: 256 5x5 filters at stride 1, pad 2

[13x13x256] **MAX POOL2**: 3x3 filters at stride 2

[13x13x256] **NORM2**: Normalization layer

[13x13x384] **CONV3**: 384 3x3 filters at stride 1, pad 1

[13x13x384] **CONV4**: 384 3x3 filters at stride 1, pad 1

[13x13x256] **CONV5**: 256 3x3 filters at stride 1, pad 1

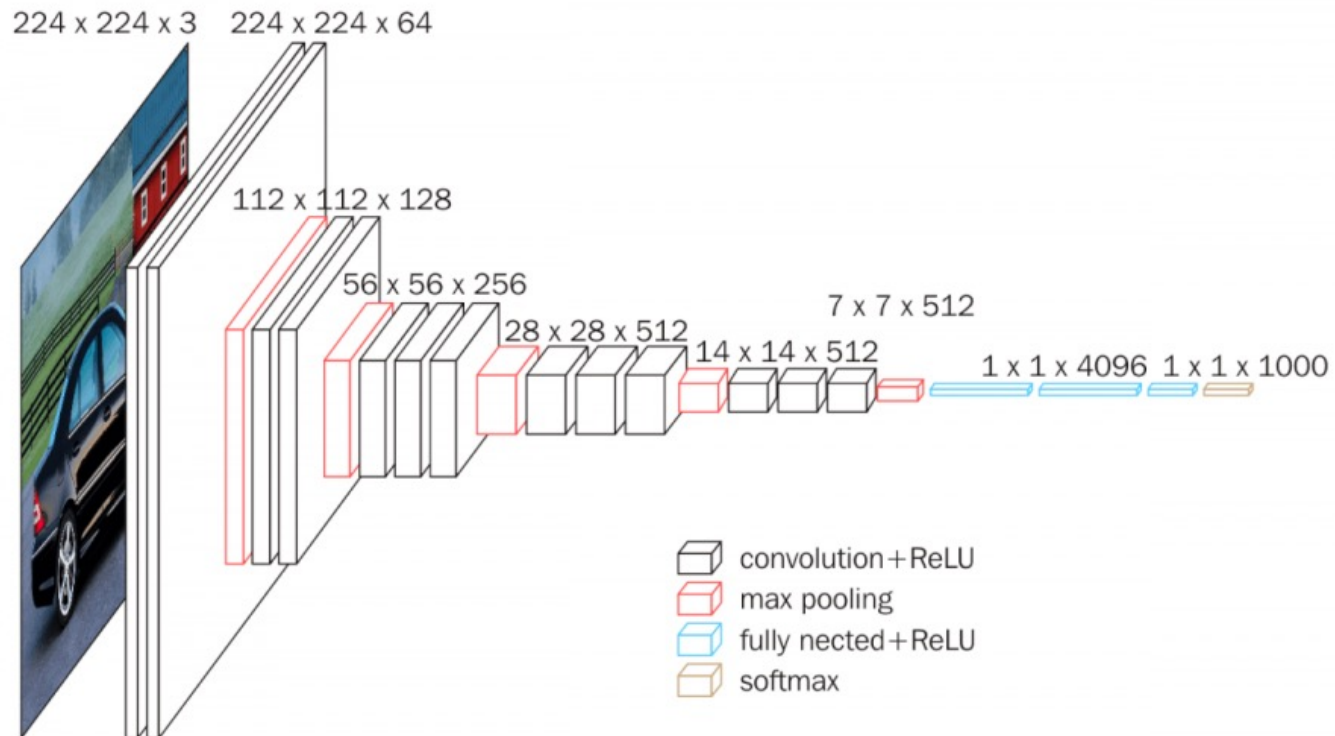
[6x6x256] **MAX POOL3**: 3x3 filters at stride 2

[4096] **FC6**: 4096 neurons

[4096] **FC7**: 4096 neurons

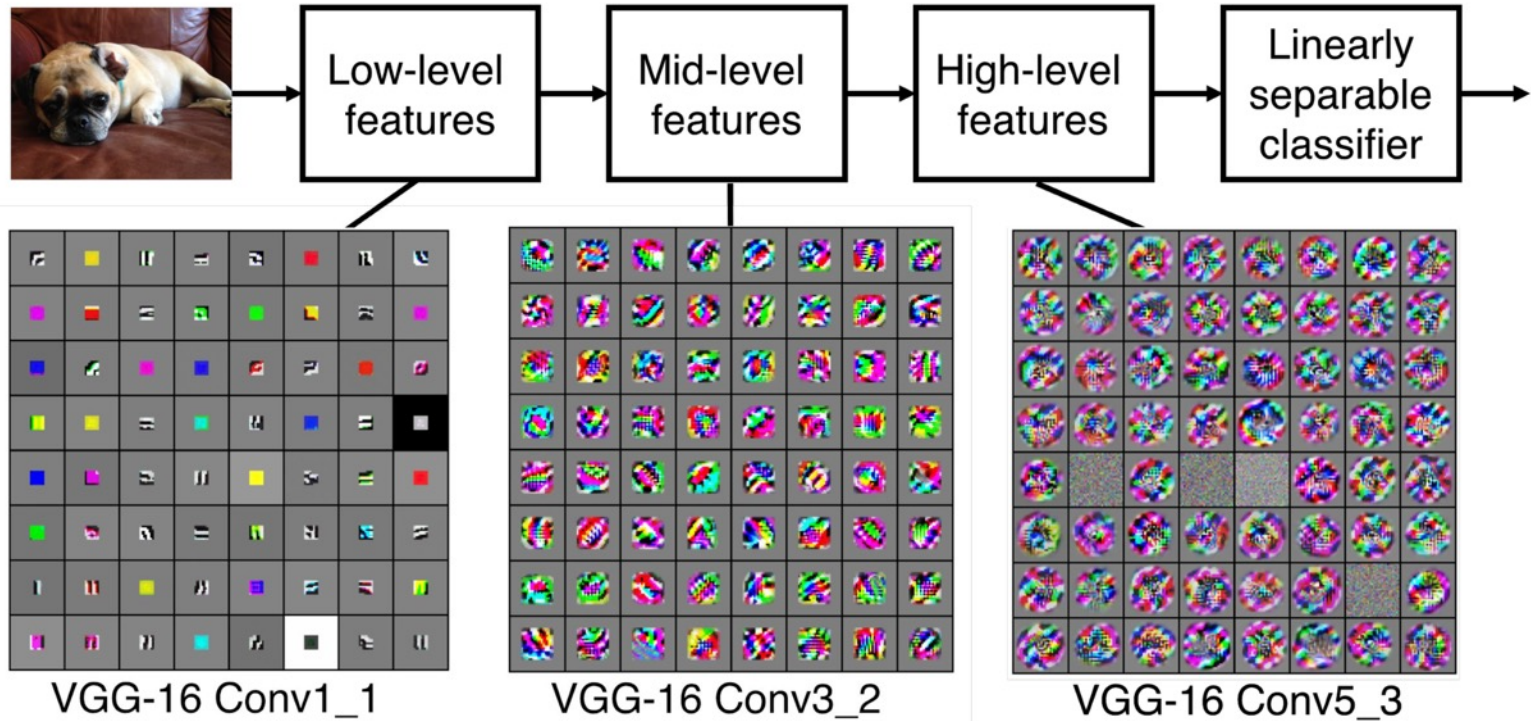
[1000] **FC8**: 1000 neurons (class scores)

VGG



What do the filters look like?

- Filters are feature extractors

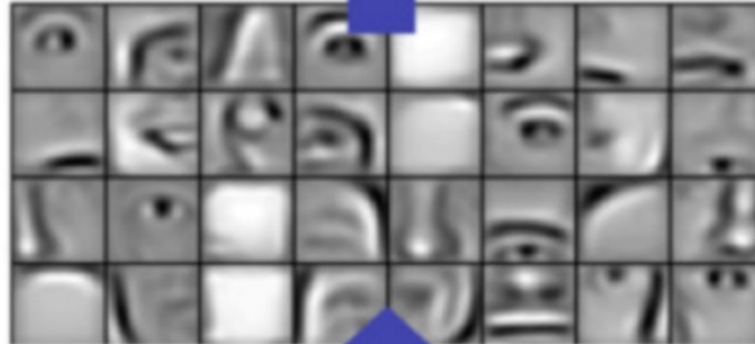


What do the filters look like?

High-level
features



Layer 3



Layer 2

Low-level
features



Layer 1

Image Source: Convolutional Deep Belief Networks for Scalable Unsupervised Learning of Hierarchical Representations, by Lee et al.

Summary

- CNN = a multi-layer neural network with convolution layers
- Convolution = local connectivity + weight sharing
- Layer types:
 - Convolutional layer (including activation function)
 - Pooling layer
 - Fully-connected layer

Today

- Machine learning
- Linear classifier
- Neural networks
- Convolutional neural networks
- **Training neural networks**
- Network architectures

Multi-layer neural network

- **Training:** find network weights $\mathbf{w}$ to minimize the error between true training labels y_i and estimated labels $f_{\mathbf{w}}(\mathbf{x}_i)$,

$$L(\mathbf{w}) = \frac{1}{n} \sum_i l(y_i, f_{\mathbf{w}}(\mathbf{x}_i))$$

For example:

- L2 loss for regression
 - Cross-entropy loss for classification
-
- Minimization can be done by gradient descent provided f is differentiable
 - This training method is called back-propagation

Training CNN with gradient descent

- A CNN as composition of functions

$$f_{\mathbf{w}}(\mathbf{x}) = f_L(\dots (f_2(f_1(\mathbf{x}; \mathbf{w}_1); \mathbf{w}_2) \dots; \mathbf{w}_L)$$

- Parameters

$$\mathbf{w} = (\mathbf{w}_1, \mathbf{w}_2, \dots \mathbf{w}_L)$$

- Loss function

$$L(\mathbf{w}) = \frac{1}{n} \sum_i l(y_i, f_{\mathbf{w}}(\mathbf{x}_i))$$

- Gradient descent

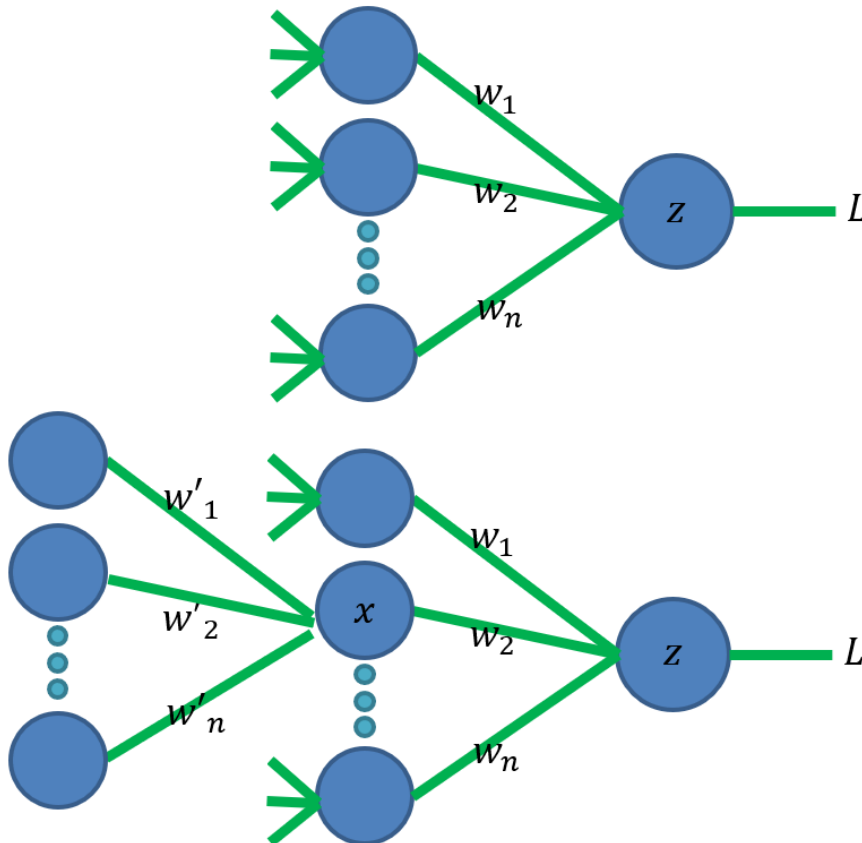
The diagram illustrates the gradient descent update rule: $\mathbf{w}^{t+1} = \mathbf{w}^t - \eta_t \frac{\partial L}{\partial \mathbf{w}}(\mathbf{w}^t)$. It features four labeled boxes with arrows pointing to the equation: a yellow box labeled 'New weight' points to $\mathbf{w}^{t+1}$; a red box labeled 'Old weight' points to $\mathbf{w}^t$; a green box labeled 'Learning rate' points to η_t ; and a blue box labeled 'Gradient' points to $\frac{\partial L}{\partial \mathbf{w}}(\mathbf{w}^t)$.

$$\text{New weight} \rightarrow \mathbf{w}^{t+1} = \mathbf{w}^t - \eta_t \frac{\partial L}{\partial \mathbf{w}}(\mathbf{w}^t)$$

Old weight Learning rate Gradient

Backpropagation

- Use chain rule to calculate gradients of loss L w.r.t. weights w

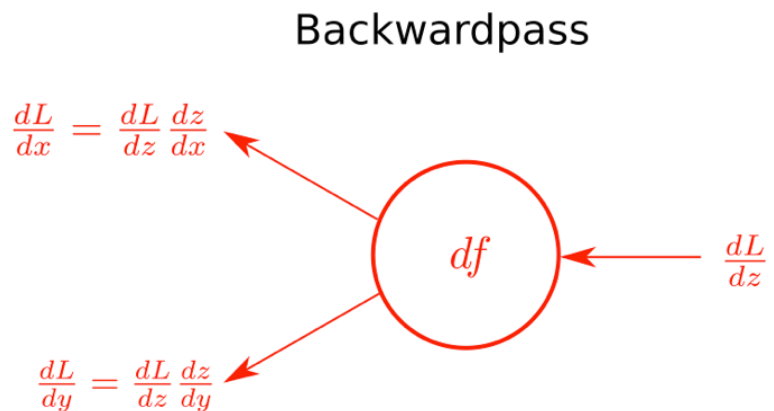
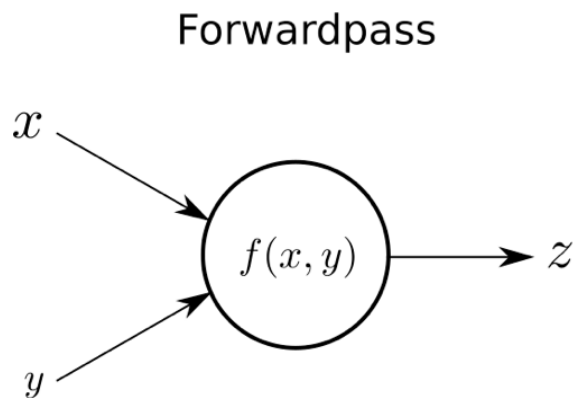


$$\frac{\partial L}{\partial w_i} = \frac{\partial z}{\partial w_i} \frac{\partial L}{\partial z}$$

$$\frac{\partial L}{\partial w'_i} = \frac{\partial x}{\partial w'_i} \frac{\partial z}{\partial x} \frac{\partial L}{\partial z}$$

Backpropagation

- Algorithm
 - 1. Forward data through the network, get loss
 - 2. Backprop to calculate the gradients
 - 3. Update the parameters using the gradient
 - 4. Go to step 1 if not converged



Stochastic gradient descent(SGD)

- Sometimes the training data is too large, e.g., millions of images in ImageNet
 - Calculate loss and gradients over all images in each iteration is too expensive
- Stochastic gradient descent: only calculate loss and gradients on a batch of randomly sampled images

$$\hat{L}(\mathbf{w}) = \frac{1}{n} \sum_{i \in \Omega} l(y_i, f_{\mathbf{w}}(\mathbf{x}_i)), \quad SG = \frac{\partial \hat{L}}{\partial \mathbf{w}}(\mathbf{w}^t)$$

- Stochastic gradients approximate the real gradients

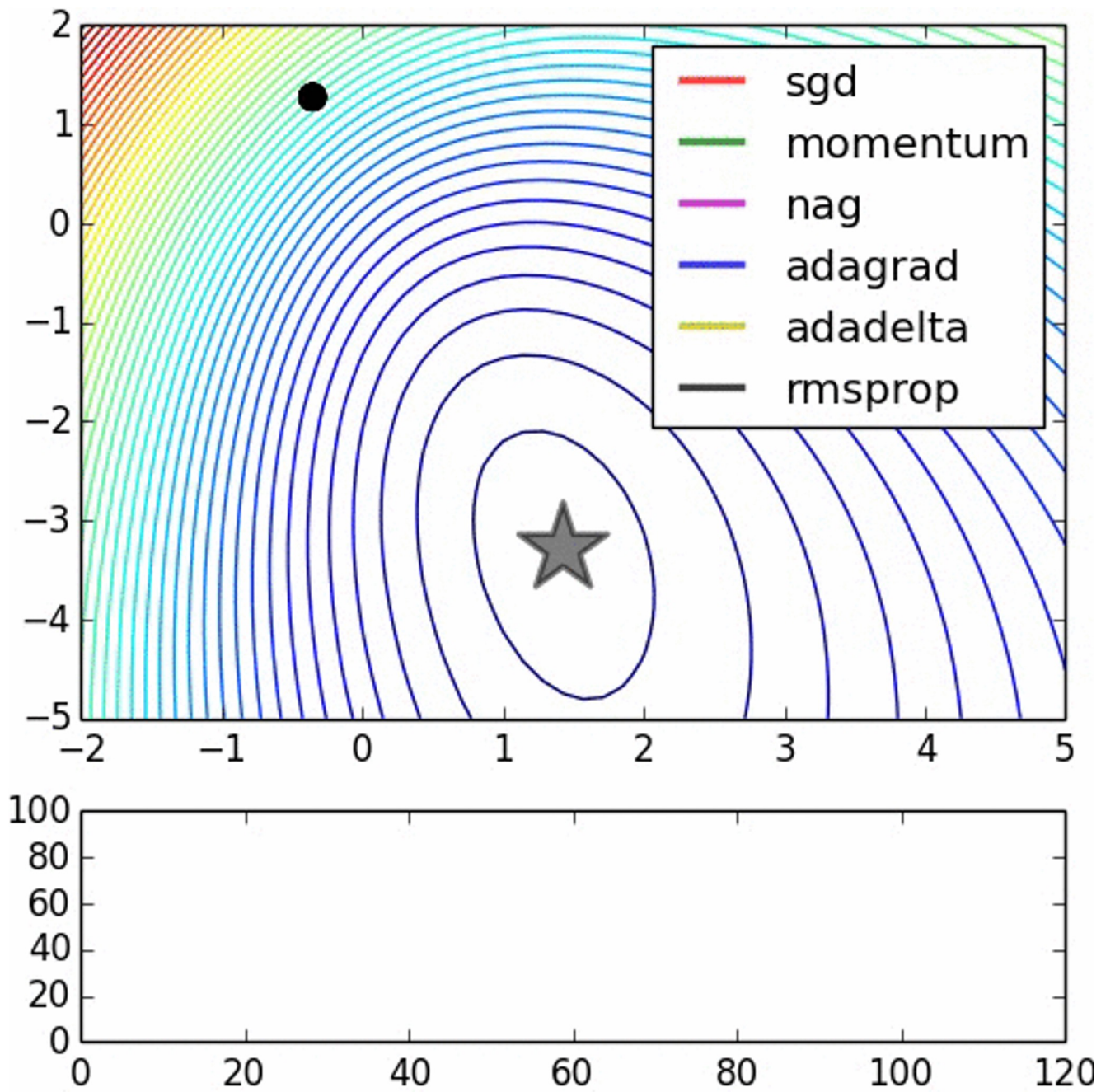


Image credits: Alec Radford

Architecture and hyper-parameters

- Hyper-parameters
 - How many layers to use?
 - How many filters in each layer?
 - What are the best batch size and learning rate?
- How do we set them?
 - One option: try them all and see what works best

Data split

Idea #1: Choose hyperparameters that work best on the data

Bad: you can always decrease training loss by using a larger network



Your Dataset

Idea #2: Split data into **train** and **test**, choose hyperparameters that work best on test data

Bad: no idea how it will perform on new data

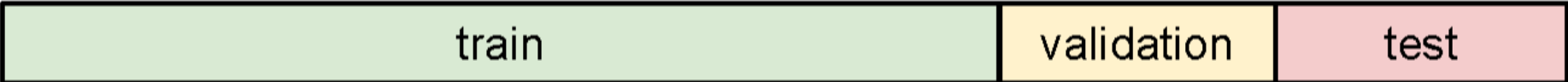


train

test

Idea #3: Split data into **train**, **val**, and **test**; choose hyperparameters on val and evaluate on test

Best! Called **cross validation**

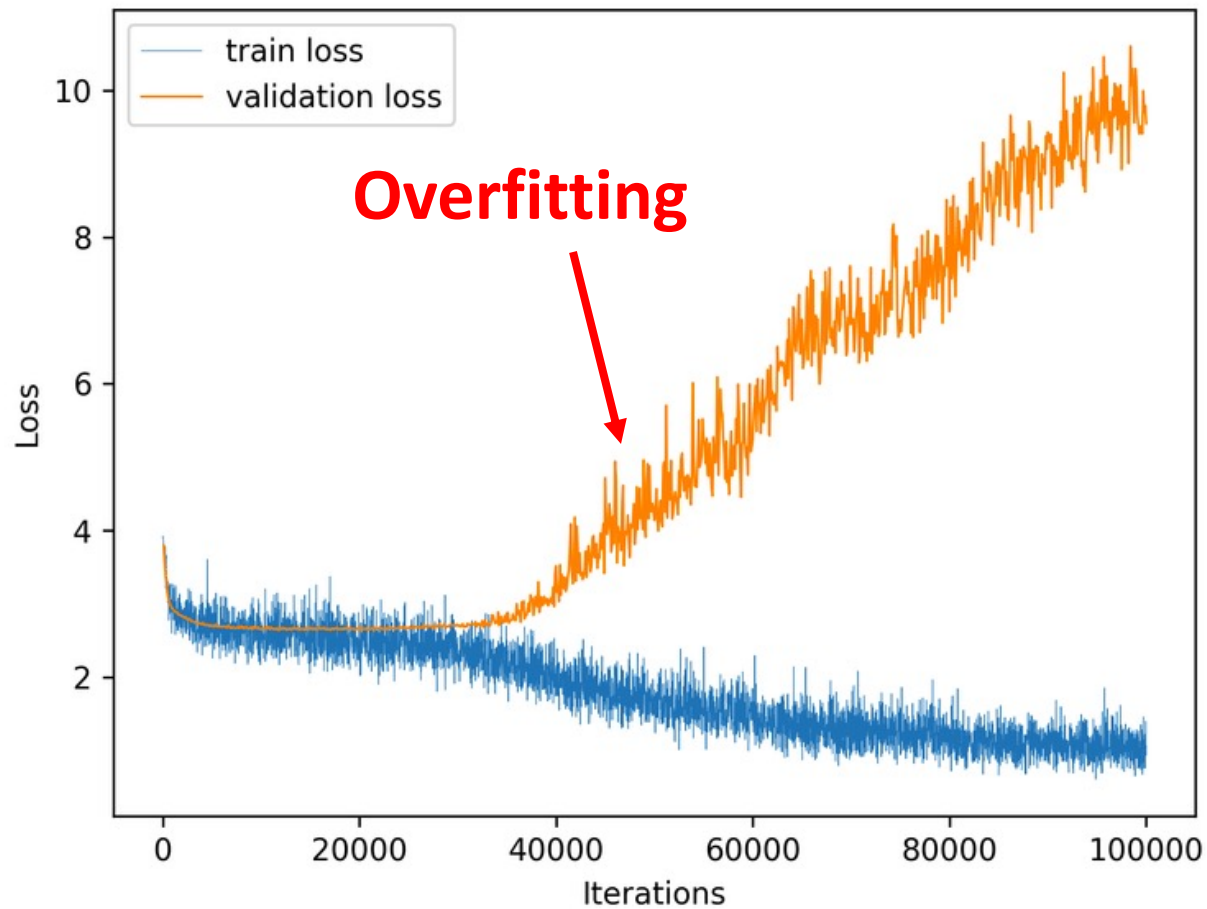


train

validation

test

Data split



How to pick hyperparameters

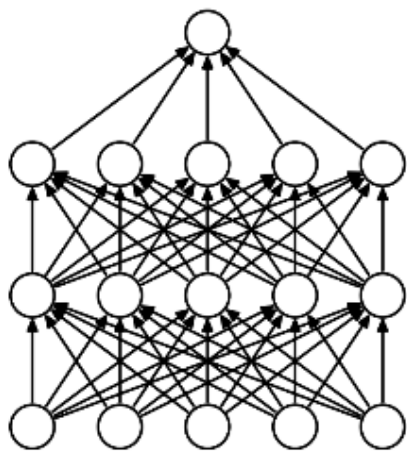
- Train for original model
- Validate to find hyperparameters
- Test to understand generalizability

Regularization

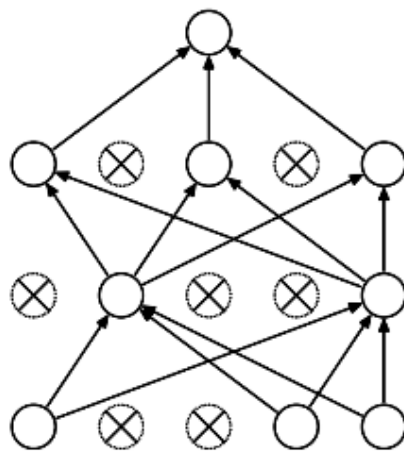
$$L = \frac{1}{N} \sum_{i=1}^N \sum_{j \neq y_i} \max(0, f(x_i; W)_j - f(x_i; W)_{y_i} + 1) + \lambda R(W)$$

- In common use:
 - L2: $R(W) = \sum_k \sum_l W_{k,l}^2$
 - L1: $R(W) = \sum_k \sum_l |W_{k,l}|$

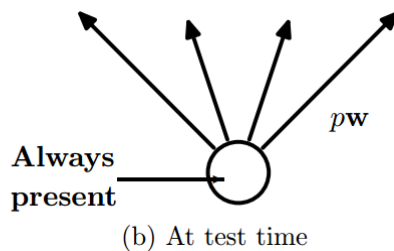
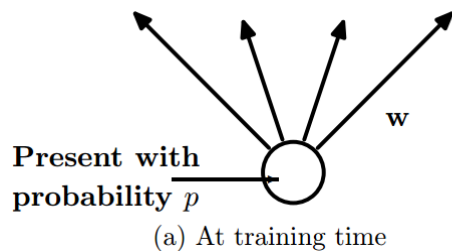
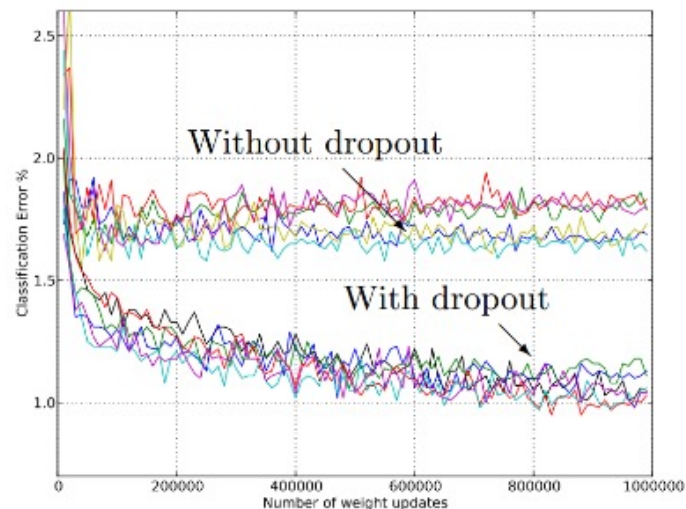
Dropout



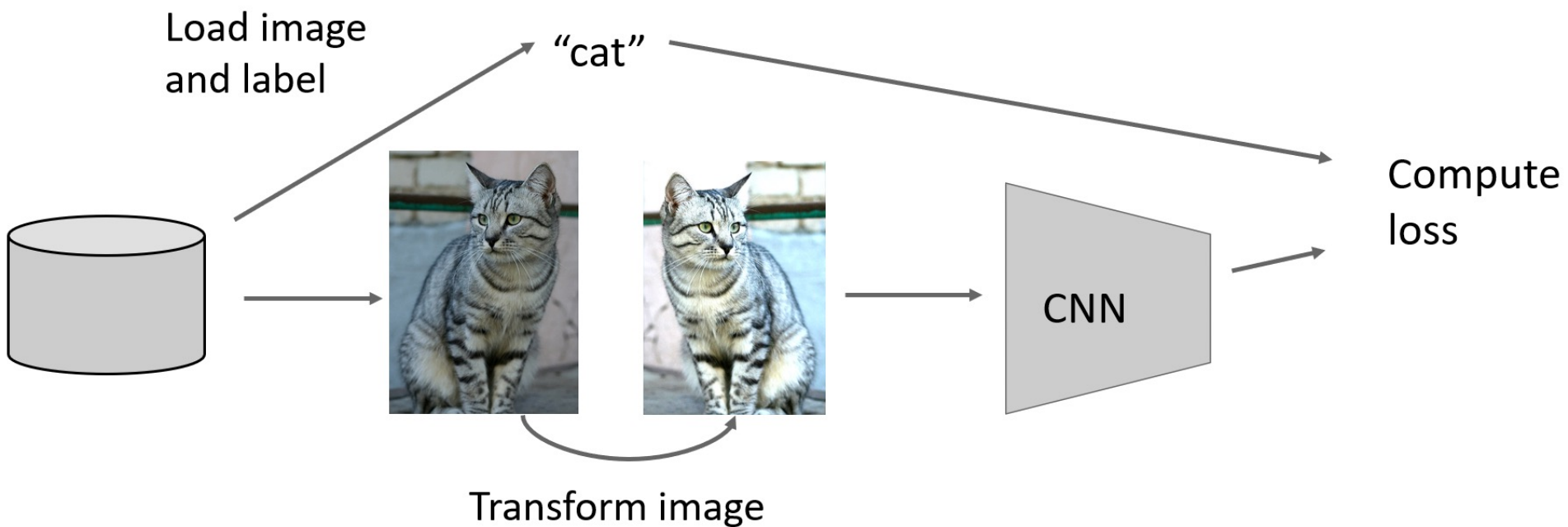
(a) Standard Neural Net



(b) After applying dropout.

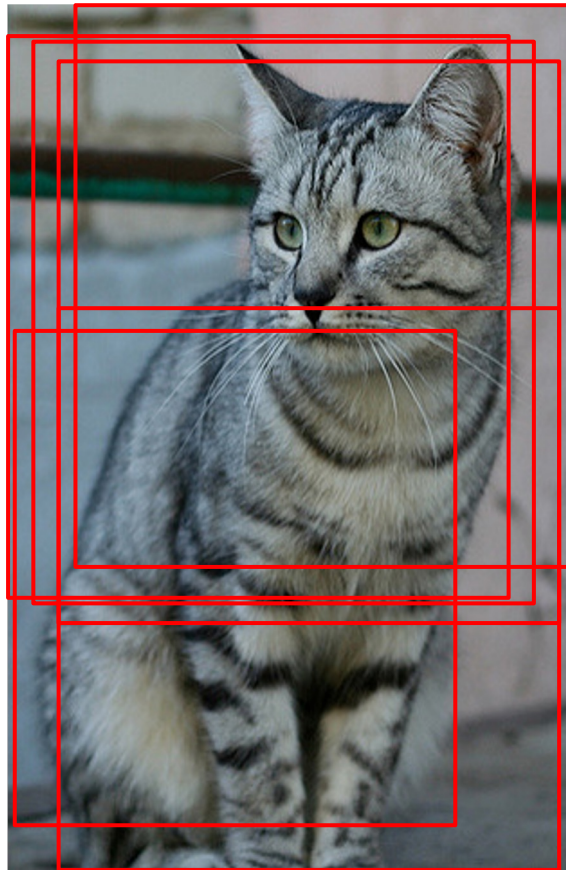


Data augmentation



Data augmentation

- Random crops and scales



How to prevent overfitting?

- Cross validation and early stop
- Regularization or dropout
- Data augmentation

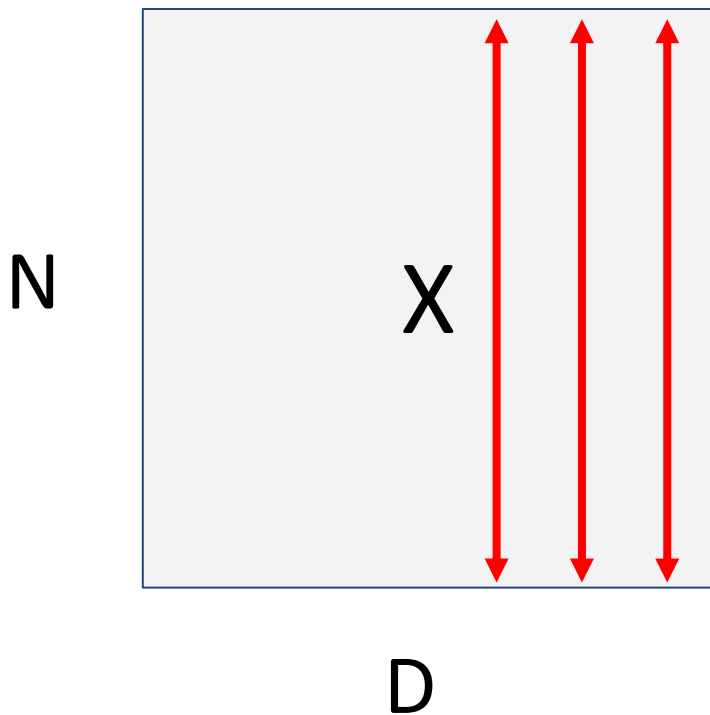
Batch Normalization

- Normalize the outputs of a layer so they have zero mean and unit variance
- Why?
 - Reduce internal covariate shift

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mathbb{E}[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$

Batch Normalization

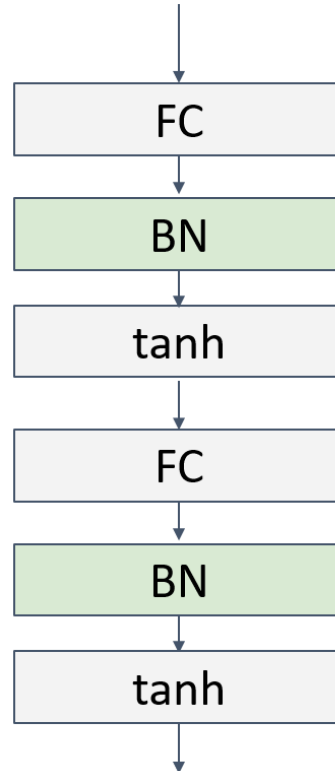
- Input: $x \in \mathbb{N} \times \mathbb{D}$
 - N is channels, D is $B \times H \times W$ for a batch of images



$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j}$$
$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$$
$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

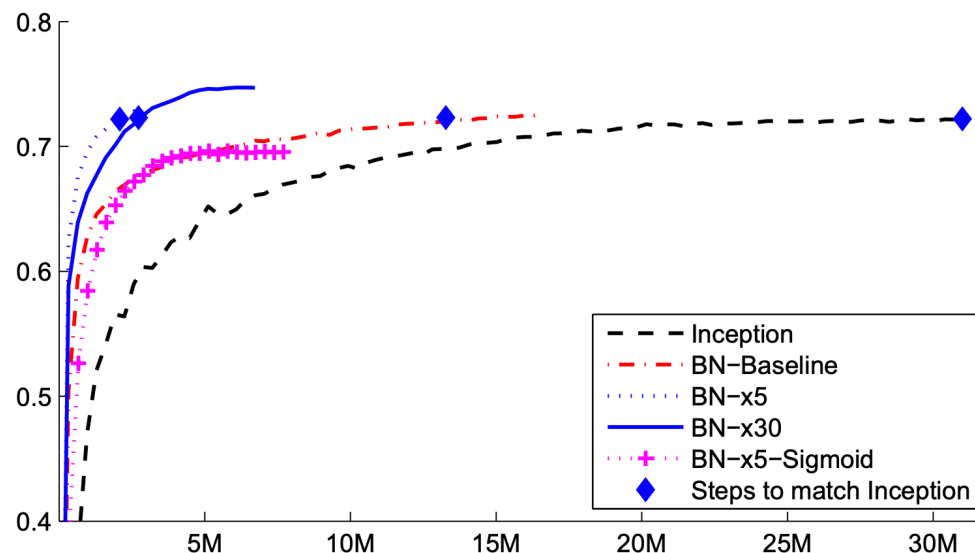
Batch Normalization

- Usually inserted after FC / Conv and before nonlinearity.



Batch Normalization

- Much easier to train
- Allow higher learning rate, faster convergence
- More robust to initialization



Deep learning frameworks

Support fast development of deep learning algorithms

- Various modules to build a network
 - Operations
 - Layers
 - Losses
- Various optimizers to train networks with CPU/GPU/Multiple GPUs

Caffe



TensorFlow

 PyTorch

Today

- Machine learning
- Linear classifier
- Neural networks
- Convolutional neural networks
- Training neural networks
- **Network architectures**

History

A bit of history...

The **Mark I Perceptron** machine was the first implementation of the perceptron algorithm.

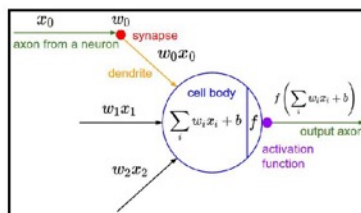
The machine was connected to a camera that used 20×20 cadmium sulfide photocells to produce a 400-pixel image.

recognized
letters of the alphabet

update rule:

$$w_i(t+1) = w_i(t) + \alpha(d_j - y_j(t))x_{j,i}$$

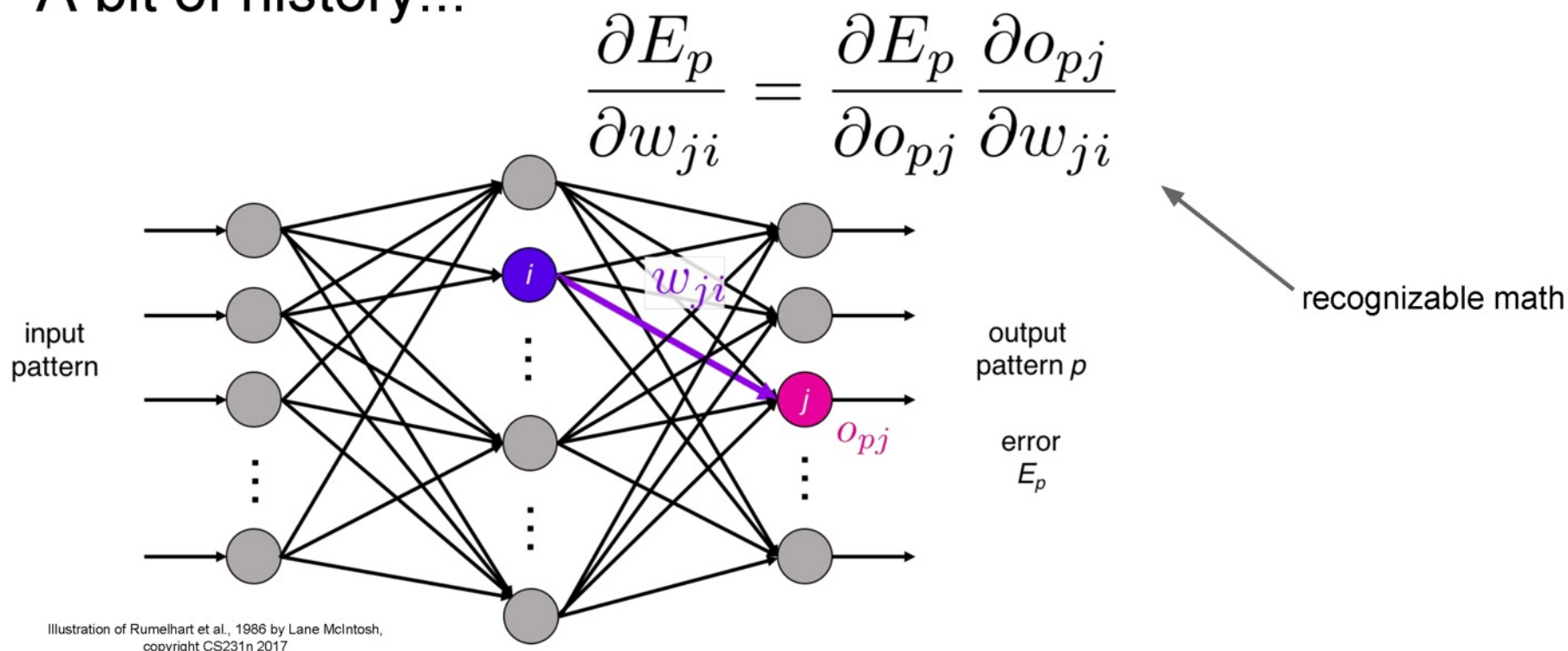
$$f(x) = \begin{cases} 1 & \text{if } w \cdot x + b > 0 \\ 0 & \text{otherwise} \end{cases}$$



Frank Rosenblatt, ~1957: Perceptron

History

A bit of history...

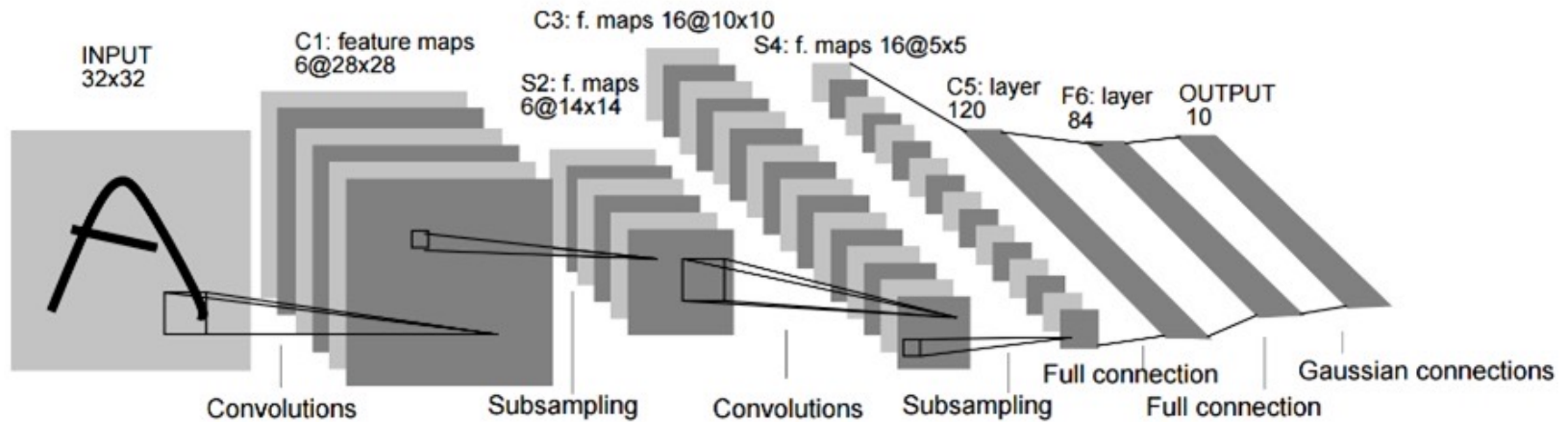


Rumelhart et al., 1986: First time back-propagation became popular

Learning representations by back-propagating errors

[David E. Rumelhart](#), [Geoffrey E. Hinton](#) & [Ronald J. Williams](#)

LeNet [LeCun et al. 1998]



Gradient-based learning applied to document recognition [[LeCun](#), [Bottou](#), [Bengio](#), [Haffner 1998](#)]

LeNet-1 from 1993

Not that successful before

- Why?
 - Small training datasets lead to overfitting
 - Hard to train a deep neural network due to limited computational power

ImageNet

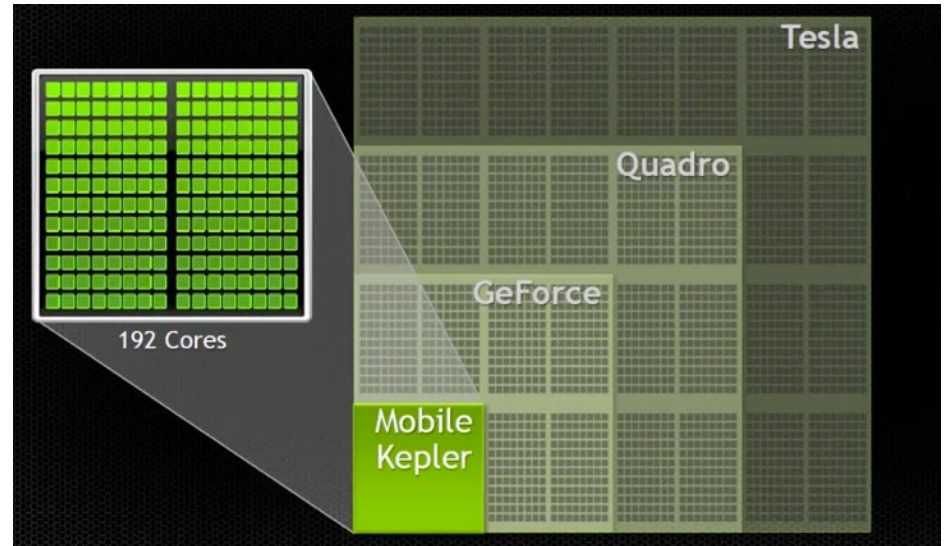


["ImageNet: A Large-Scale Hierarchical Image Database"](#)

Deng, Jia; Dong, Wei; Socher, Richard; Li, Li-Jia; Li, Kai; Fei-Fei, Li
CVPR 2009

GPU

- Graphics Processing Unit



AlexNet

- Milestone

ImageNet Classification with Deep Convolutional Neural Networks *[Krizhevsky, Sutskever, Hinton, 2012]*

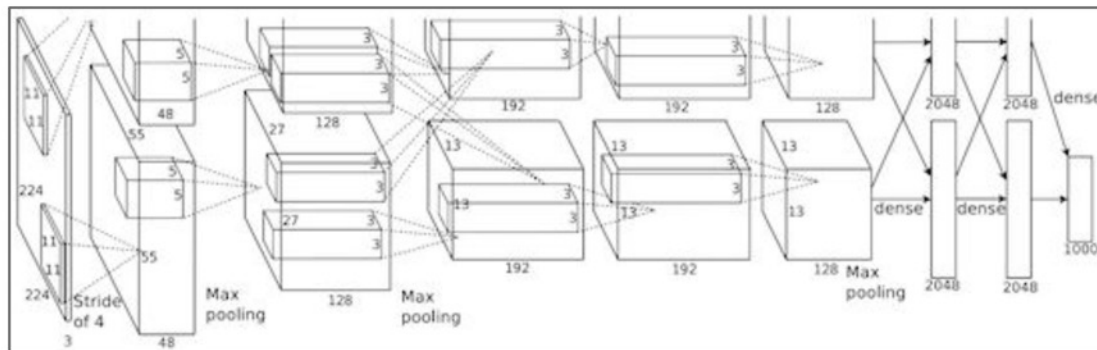
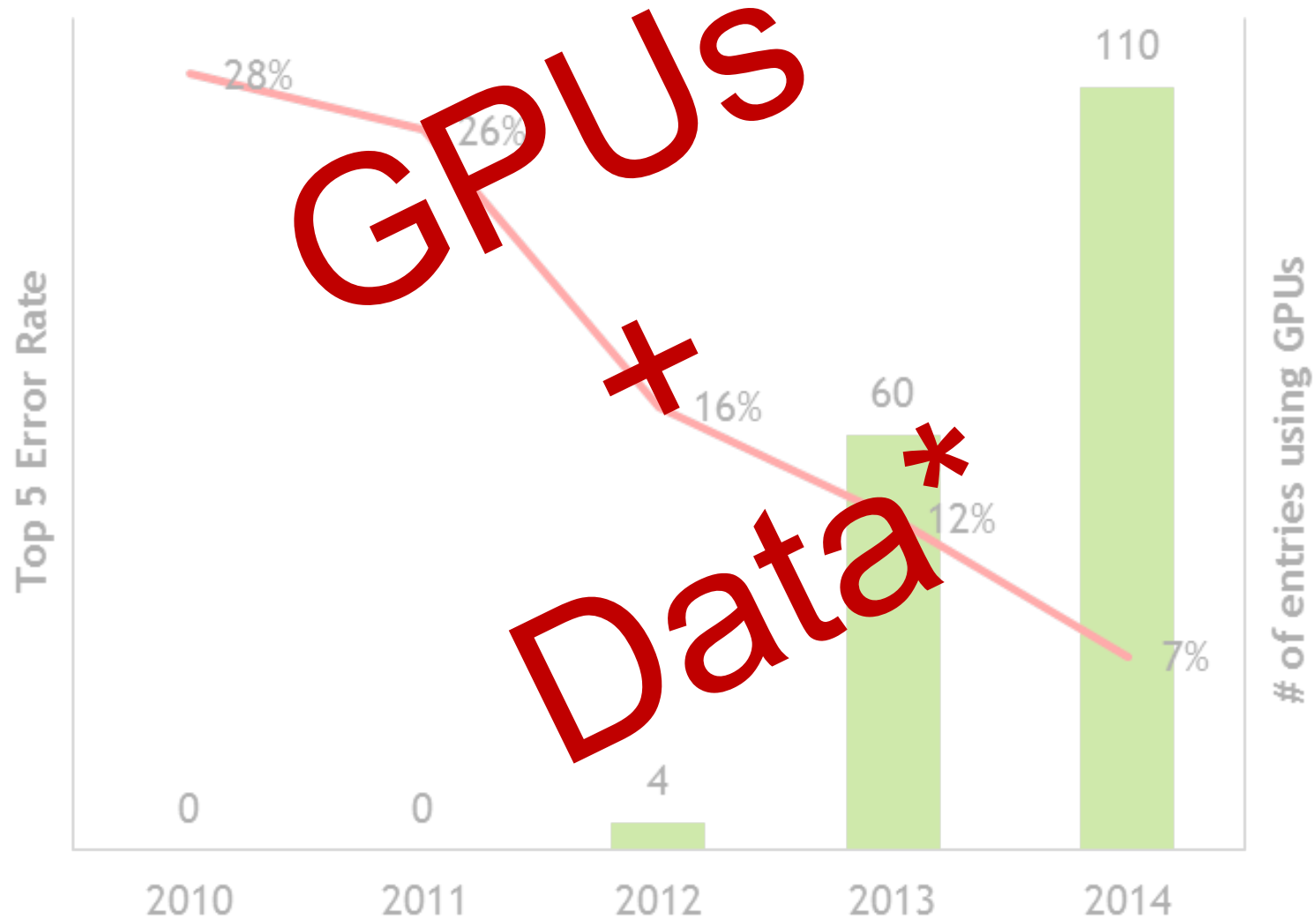


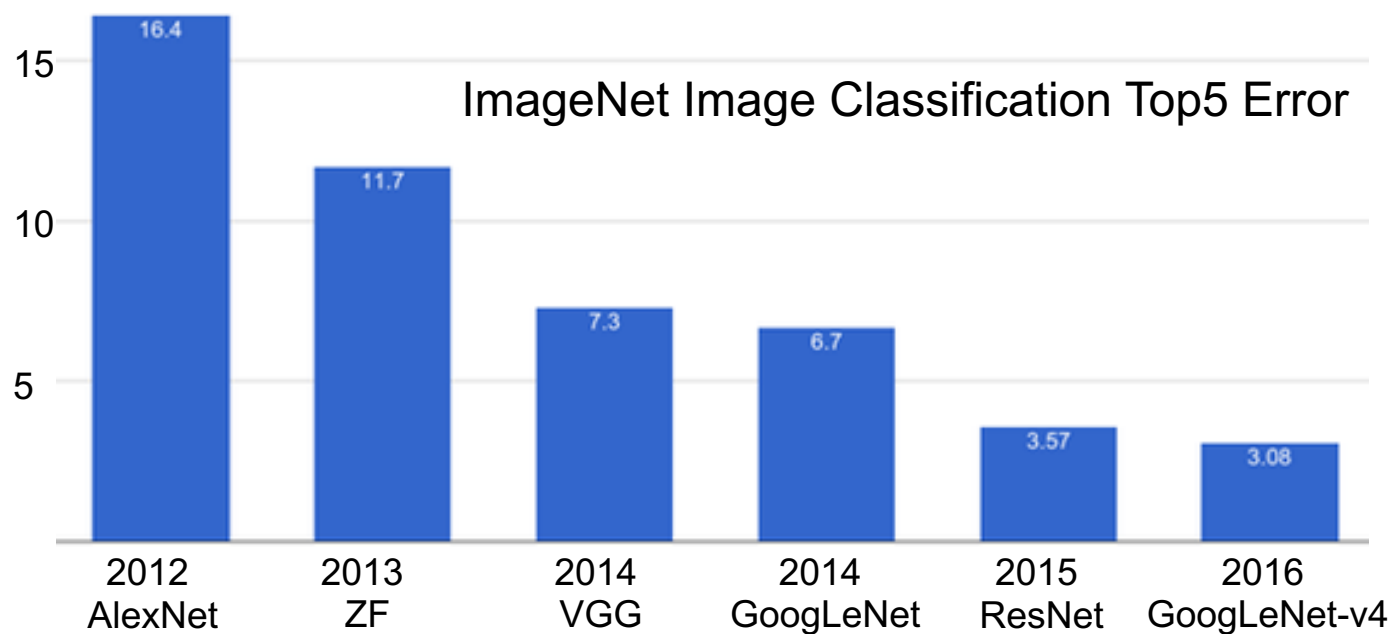
Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

“AlexNet”

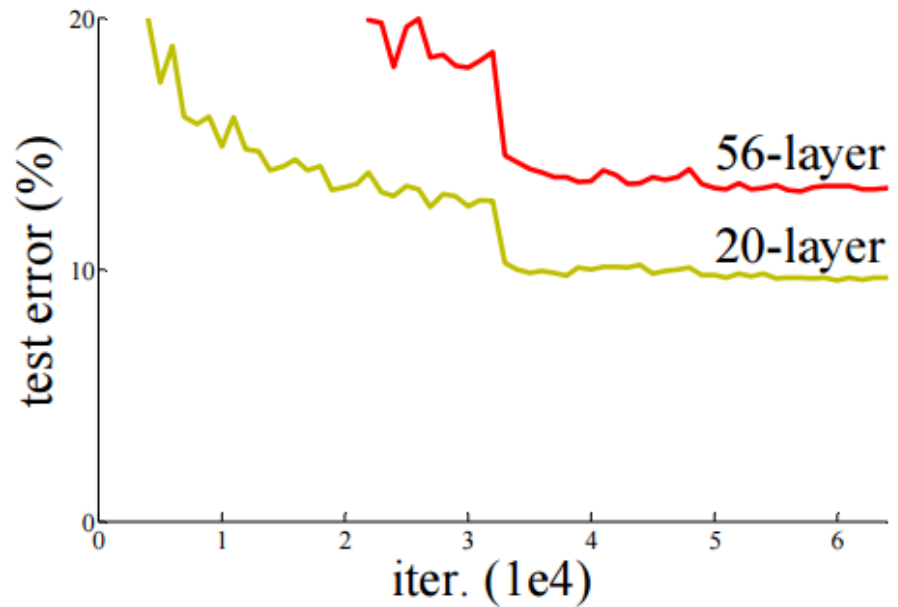
IMAGENET



Progress on ImageNet

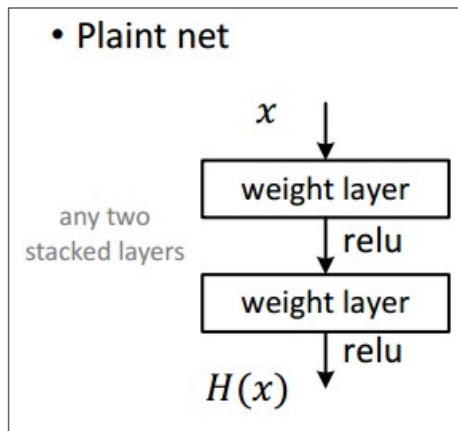


Can we just increase layers?



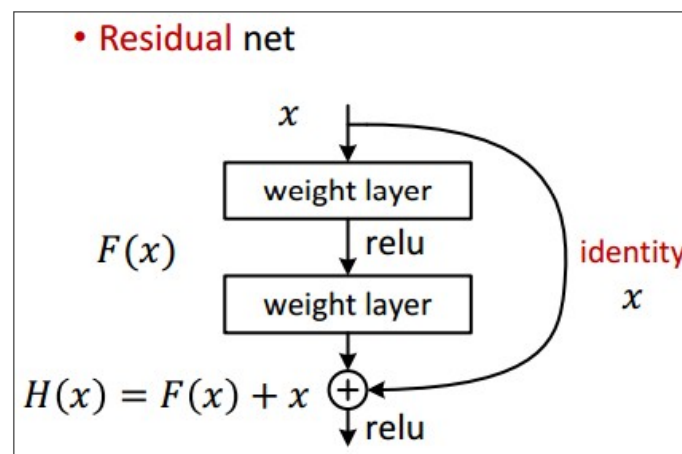
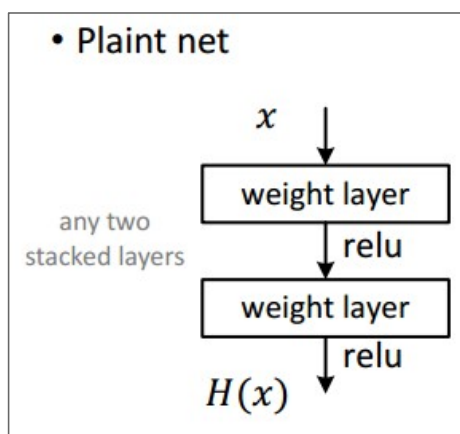
ResNet

- Why adding a layer increases error?
 - Observation – identity map is not easy to learn



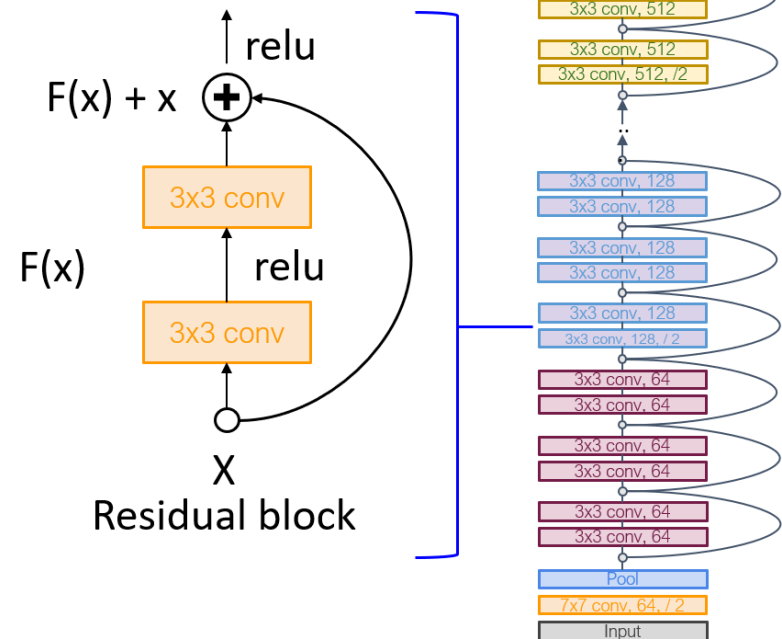
ResNet

- Residual learning (He et al., 2015)
 - Change the network so learning identity functions with extra layers is easy



ResNet

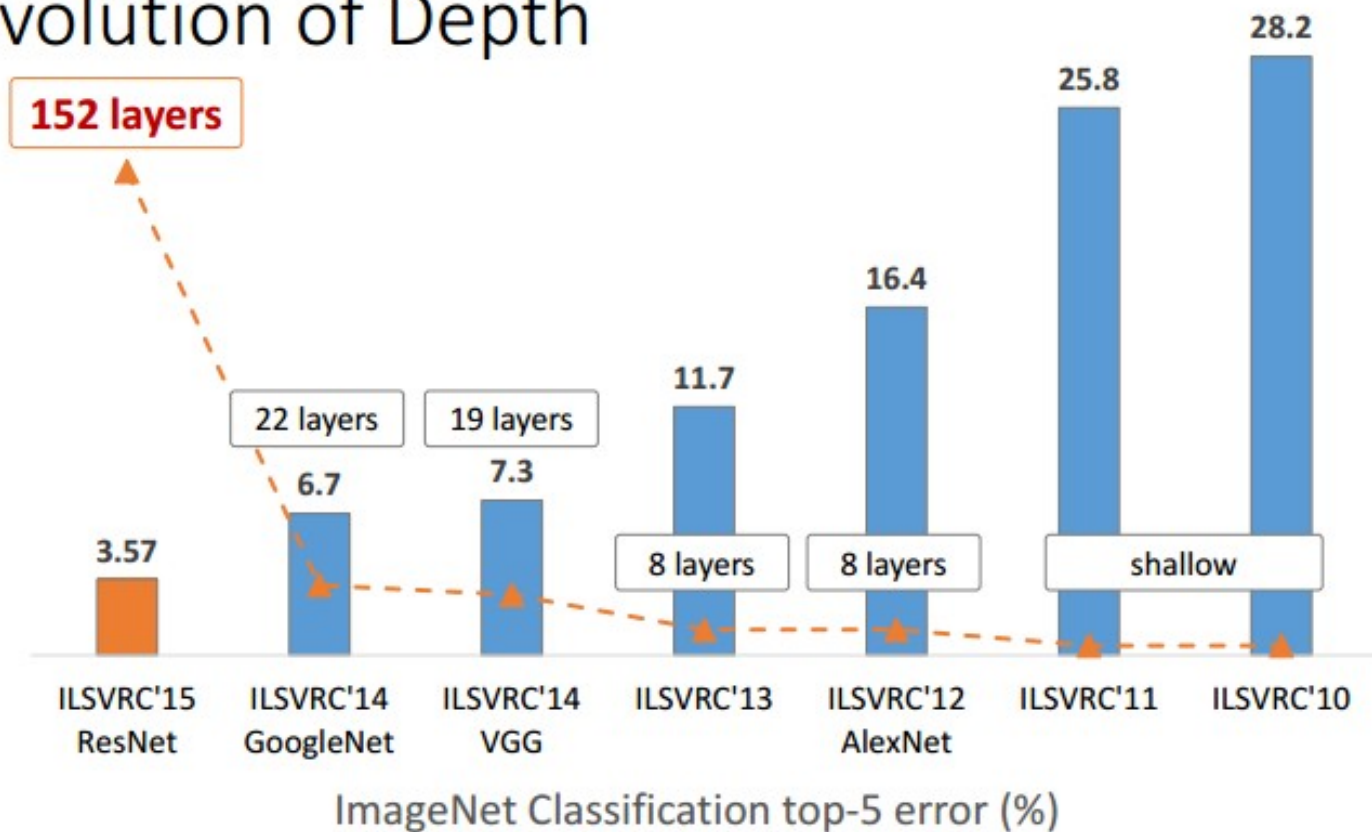
- A stack of many residual layers
 - suppose $f(x) = x + \Delta x_1 + \Delta x_2 + \Delta x_3 + \dots$
 - Only make each layer to learn Δx_i



ResNet

Microsoft
Research

Revolution of Depth



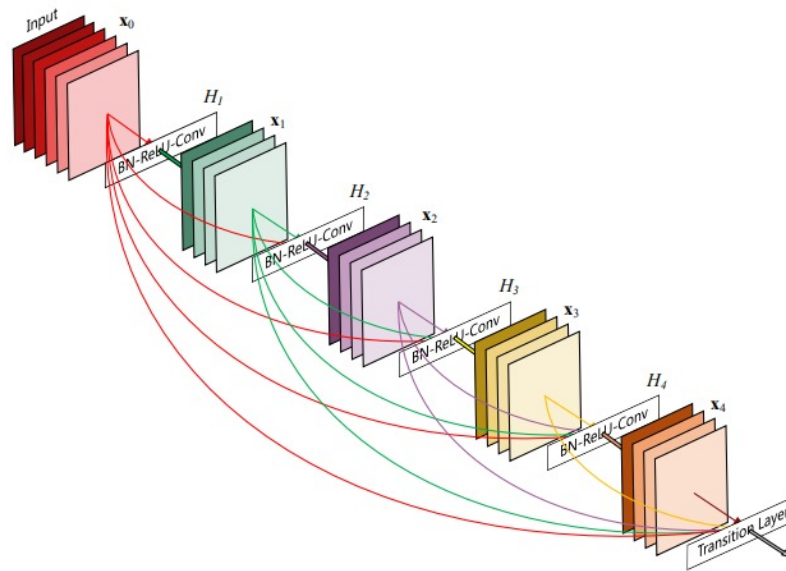
ImageNet Classification top-5 error (%)

Kaiming He, Xiangyu Zhang, Shaoqing Ren, & Jian Sun. "Deep Residual Learning for Image Recognition". arXiv 2015.

(slide from Kaiming He's recent presentation)

DenseNet

- Dense blocks where each layer is connected to every other layer in feedforward fashion [Huang, Gao, et al. CVPR'16]
- Alleviates vanishing gradient, strengthens feature propagation, encourages feature reuse

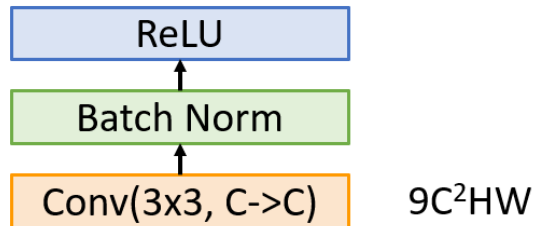


MobileNets

- Tiny networks for mobile devices

Standard Convolution Block

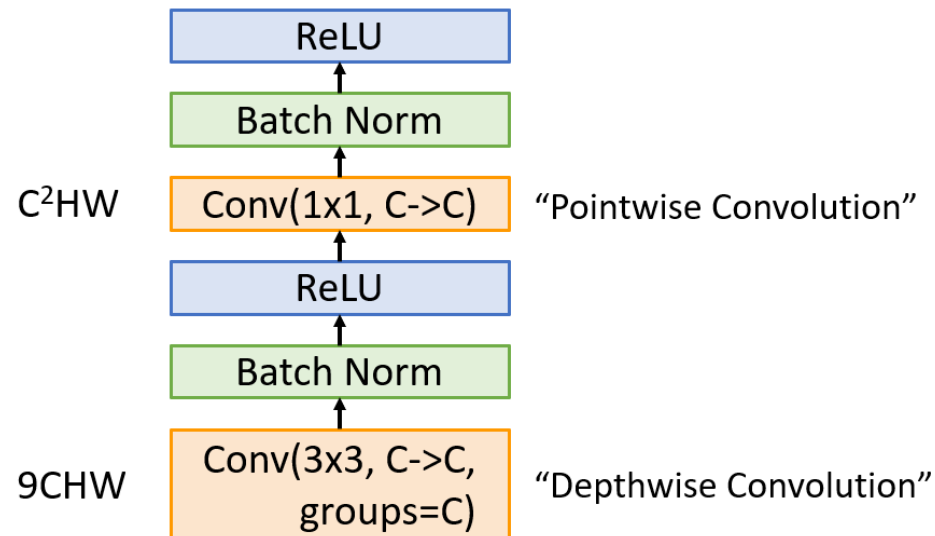
Total cost: $9C^2HW$



$$\begin{aligned}\text{Speedup} &= 9C^2 / (9C + C^2) \\ &= 9C / (9 + C) \\ &\Rightarrow 9 \text{ (as } C \rightarrow \infty)\end{aligned}$$

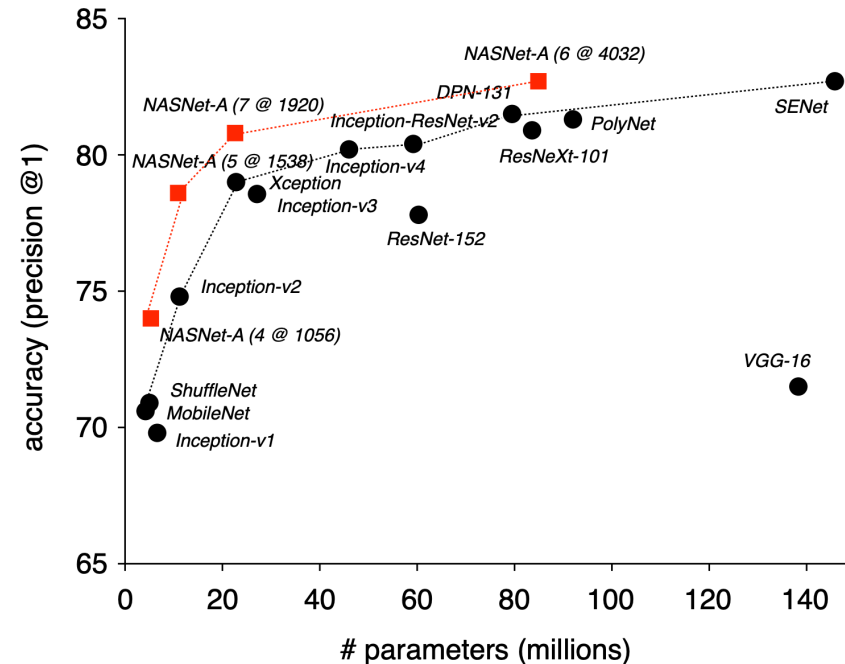
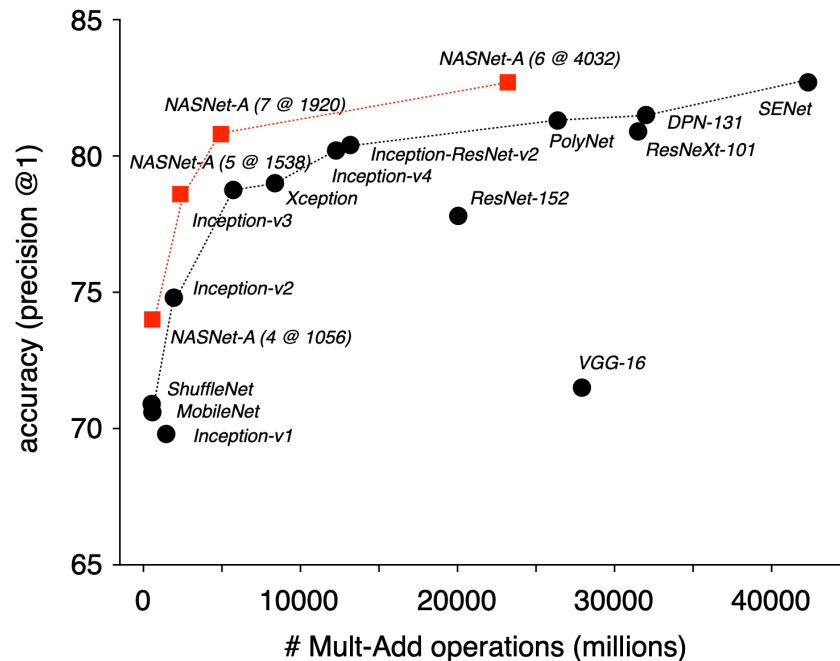
Depthwise Separable Convolution

Total cost: $(9C + C^2)HW$



Neural architecture search

- Designing a network is hard, let's automate it!
 - Very expensive



Questions?

Summary

- What is machine learning
- What is deep learning
- How to use deep learning to solve problems

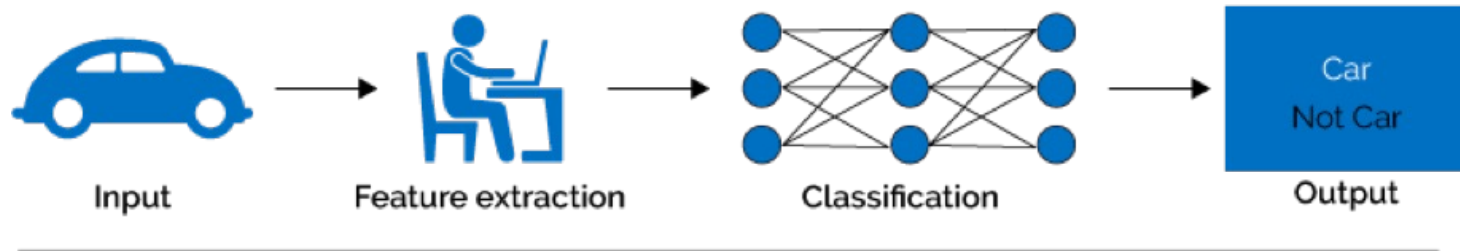
Once again: General pipeline for solving a problem with machine learning

- Define the problem
 - What is x and y
- Collect training data: (x,y) pairs
- Design a model
 - What is the form of f_W (design a network)
- Model training
 - Define a loss function $l(w)$
 - Find optimal w that minimizes $l(w)$ using some optimization algorithm (deep learning frameworks will do this)
- Testing (application)
 - Given a new data x , predict y with the learned f_W

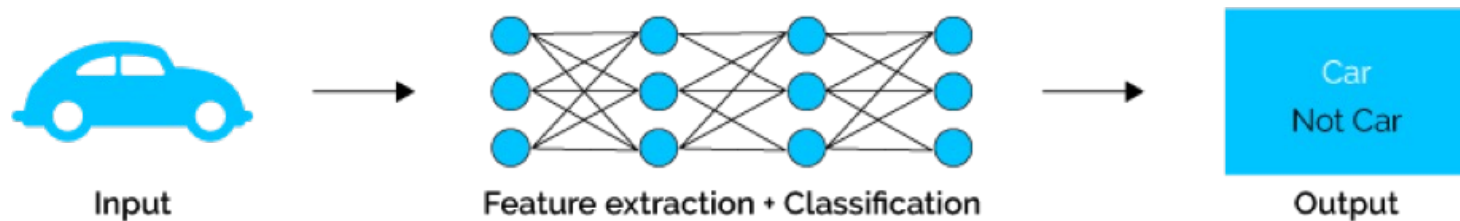
Why deep learning is powerful

- End-to-end learning

Traditional Machine Learning



Deep Learning



Why deep learning is successful

- More data
 - e.g. ImageNet, ShapeNet
- Better hardwares
 - e.g. GPU
- Better models/algorithms
 - e.g. CNN, ResNet, ...

Why deep learning is popular

- Widely applicable
 - Neural network is a general function approximator
- Easy to use
 - Many good libraries, e.g., PyTorch, TensorFlow
 - Don't need to build a model from scratch
- AI is everywhere



2019 Turing Awards



Yann LeCun



Geoffrey Hinton



Yoshua Bengio